

Join the lecture online on your dashboard.

CNN2: Evolution of CNNs

WE'LL COVER: NEURAL NETWORK ARCHITECTURES

1. Upsampling (Transposed Conv)

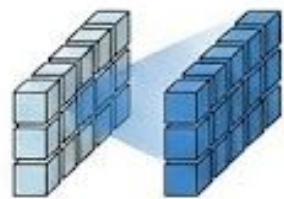
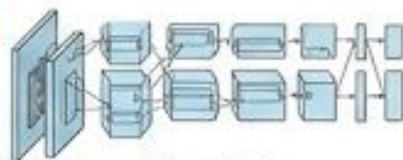


Image Restoration

2. Basic Architectures: AlexNet/VGG



AlexNet



VGG-16

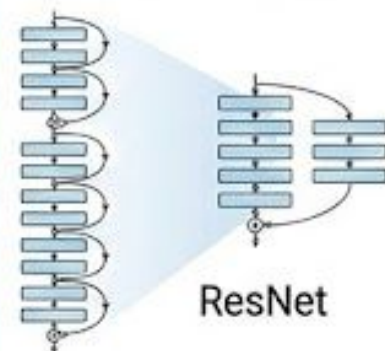
3. Residual Blocks

Skip Connection

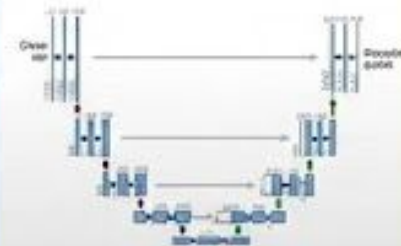


Skip Connection

4. Resnets and U-nets



ResNet

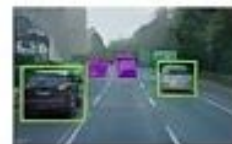


U-Net

5. Applications



Medical Imaging
(Segmentation)



Autonomous Driving
(Object Detection)

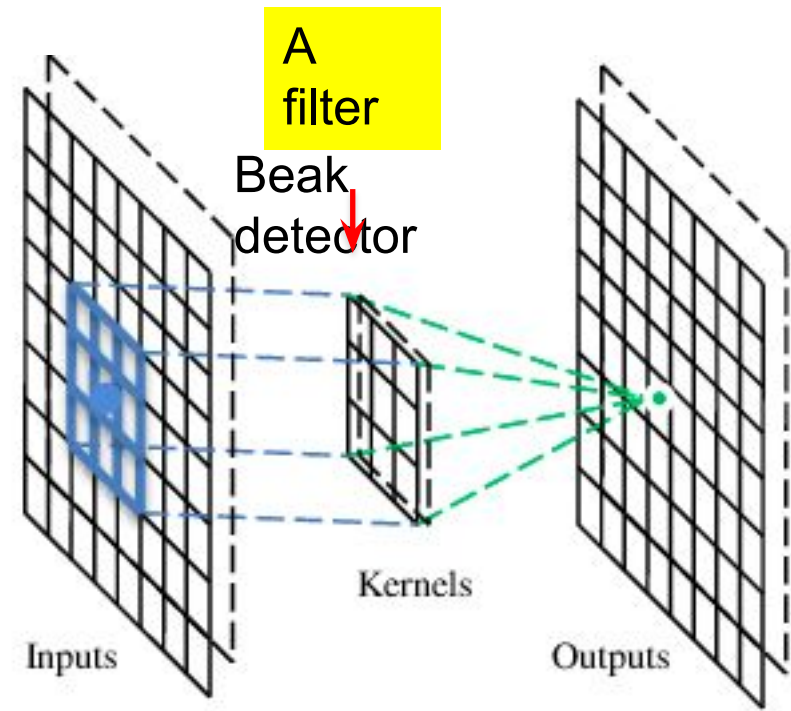


Image Generation
(GANs)

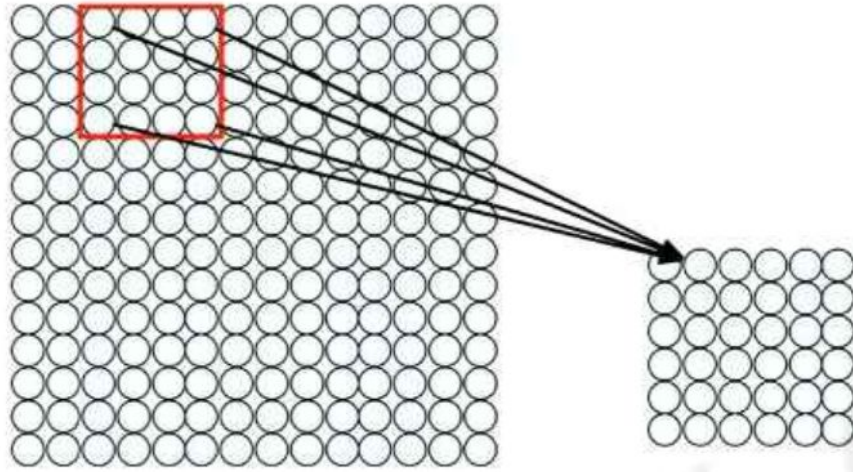
Recap

A convolutional layer

- A CNN is a neural network with some convolutional layers (and some other layers).
- A convolutional layer has a number of filters that does convolutional operation.



Feature Extraction with Convolution

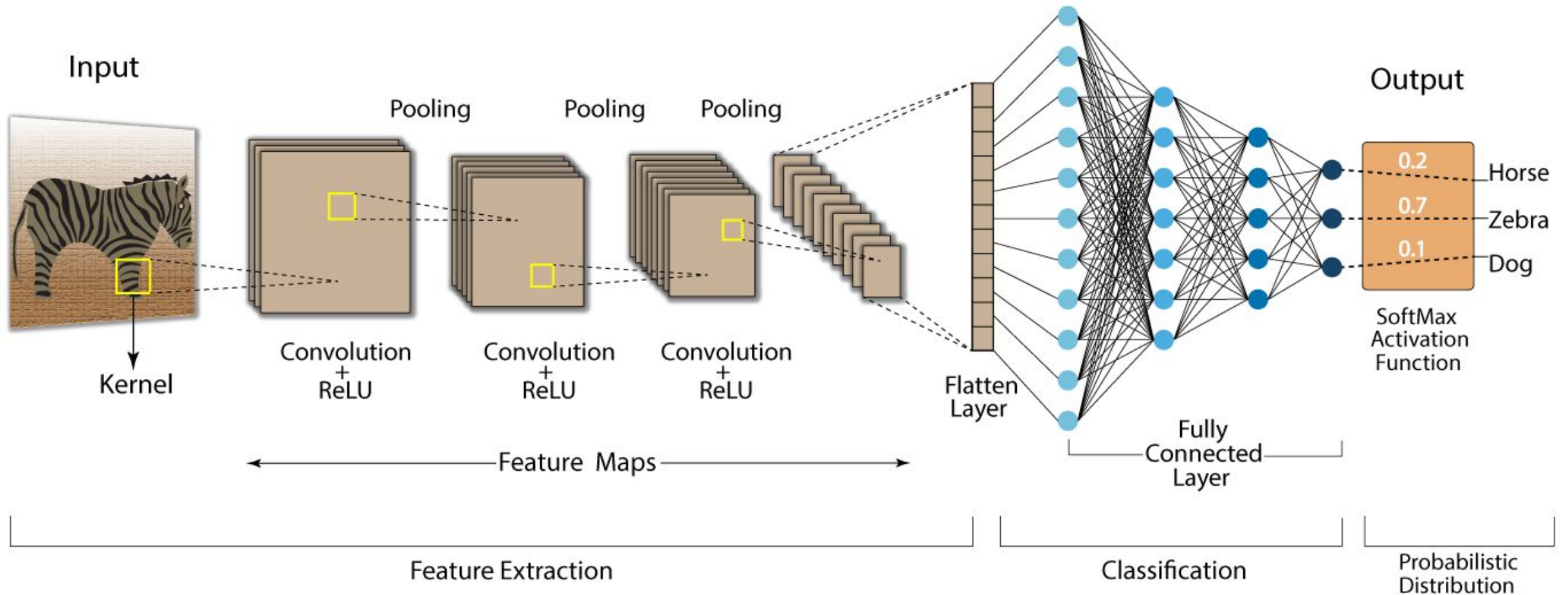


- Filter of size 4x4 : 16 different weights
- Apply this same filter to 4x4 patches in input
- Shift by 2 pixels for next patch

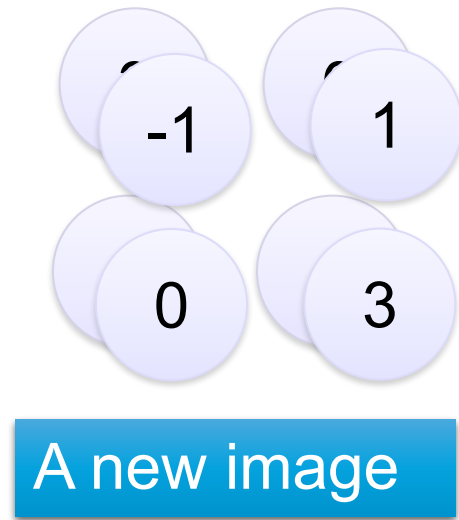
This “patchy” operation is **convolution**

- 1) Apply a set of weights – a filter – to extract **local features**
- 2) Use **multiple filters** to extract different features
- 3) **Spatially share** parameters of each filter

Convolution Neural Network (CNN)

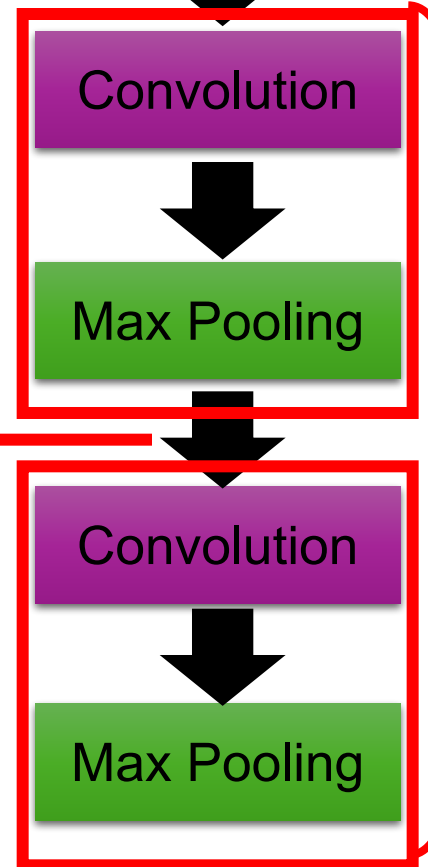


CNN Architecture



Smaller than the original image

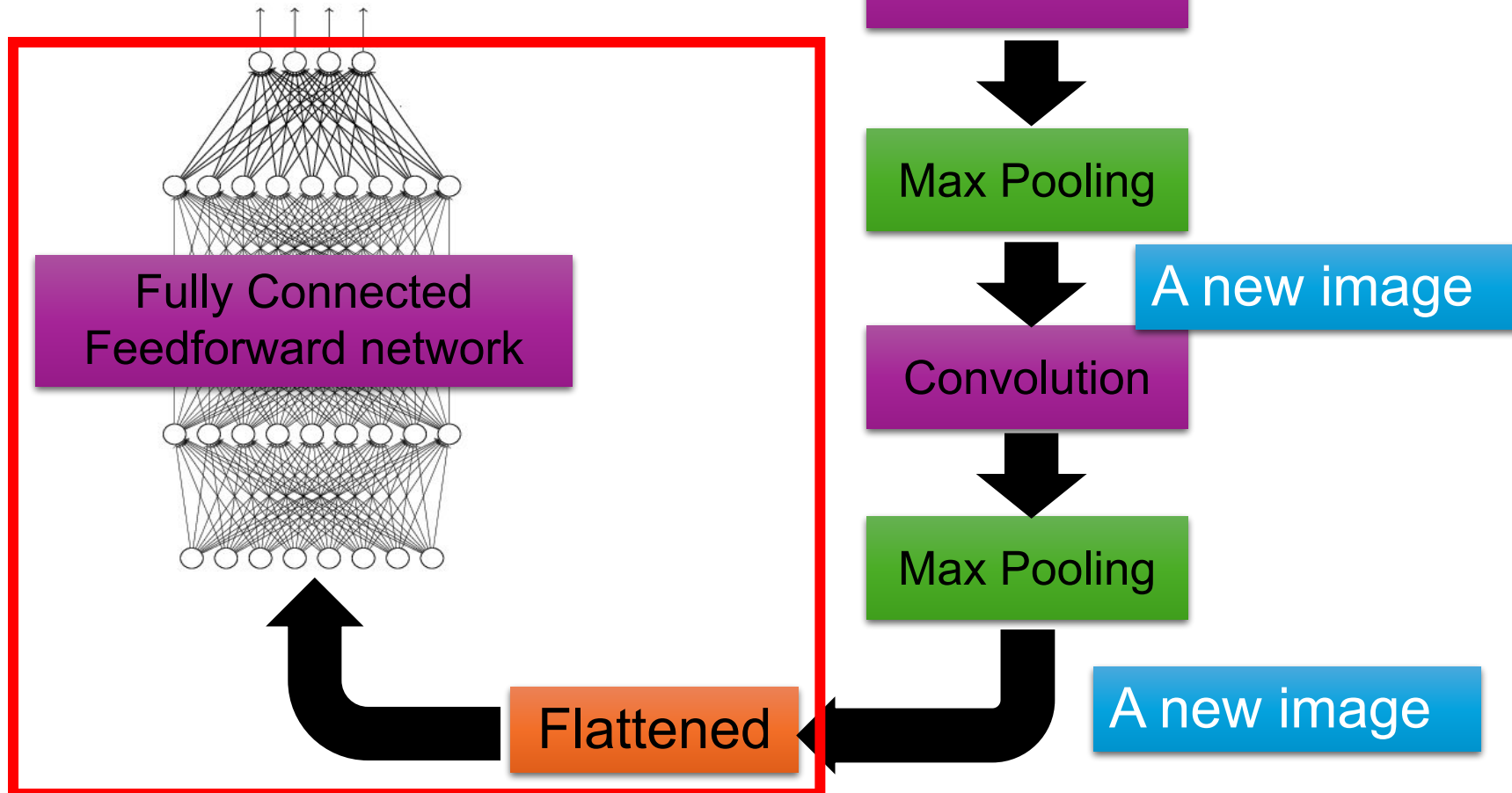
The number of channels is the number of filters



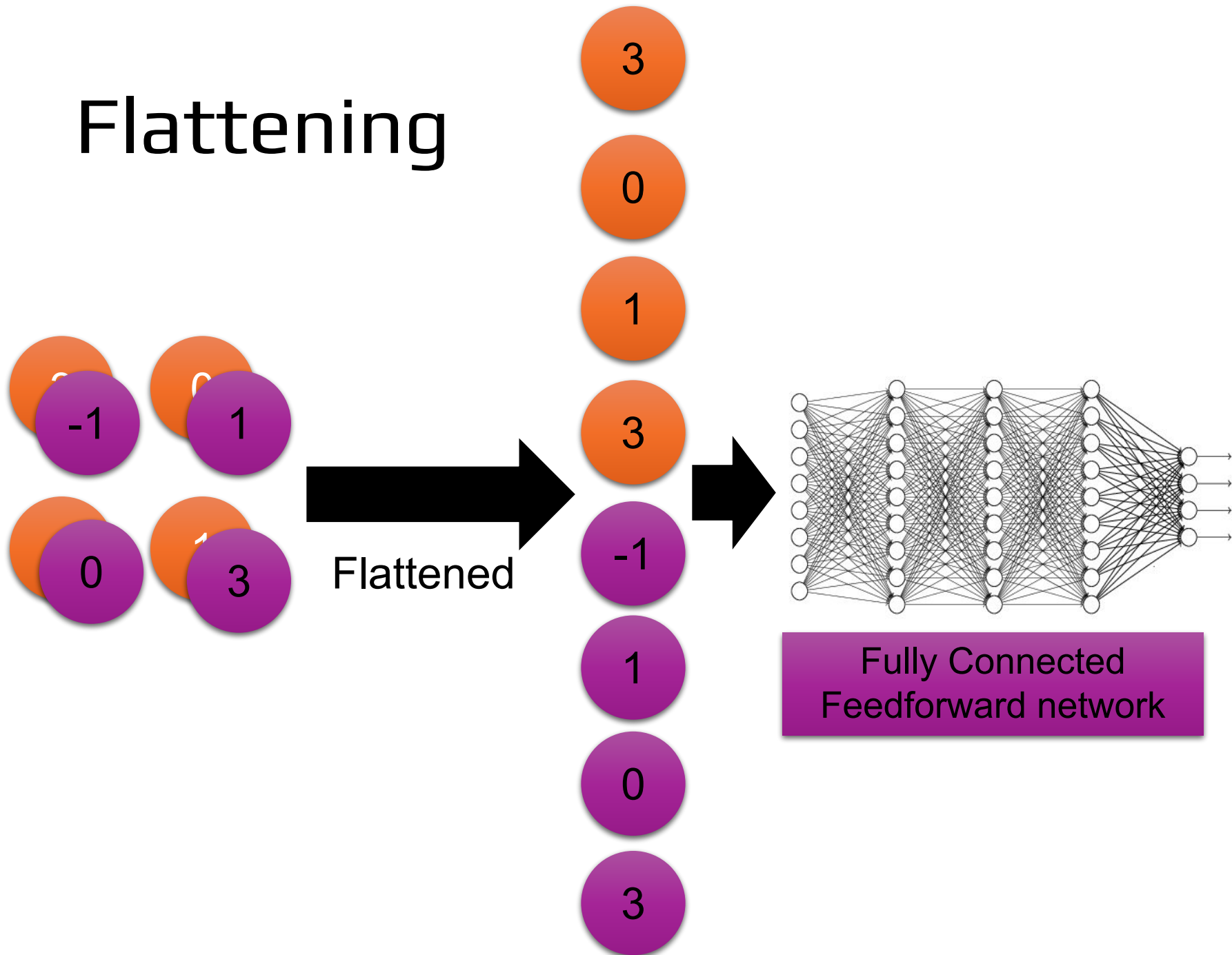
Can repeat many times

CNN Architecture

cat dog



Flattening



Kernel size, Stride, Dilation

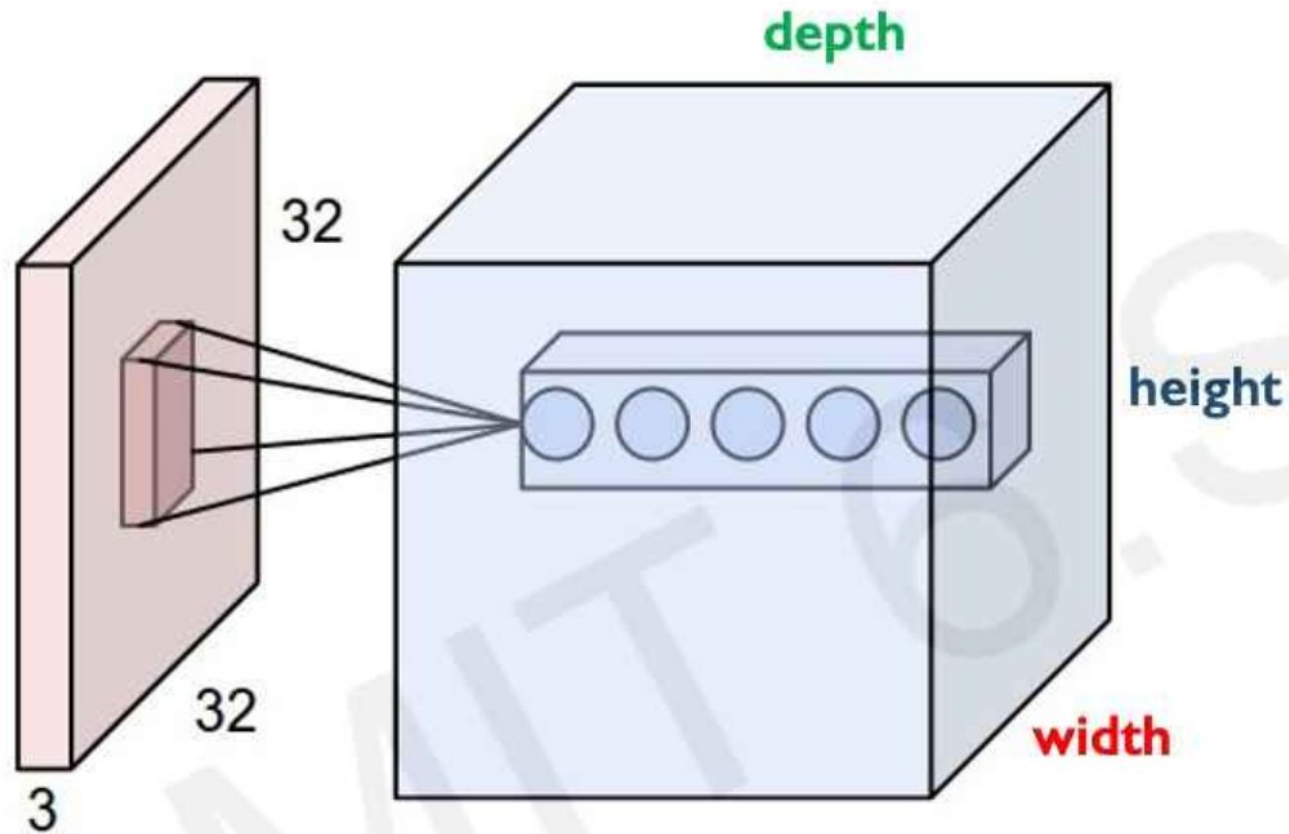
- **Kernel** size is the size of input region for which weighted sum is calculated.
- **Stride** is the amount of shift between subsequent convolution.
- **Dilation** rate is determined by the number of 0s interspersed between kernel weights.
 - Dilation rate of 1 is the normal convolution.
 - Dilation rate of 2 skips 1 pixel (i.e. insert one 0 between kernel weights).

Output volume

- $O = \frac{W - K + 2P}{S} + 1$

- W: Input size (Width or height)
- K: Kernel or Filter size
- P: Padding
- S: Stride
- The depth of the output volume is equal to number of Filters N.
- Final volume: $O \times O \times N$

CNNs: Spatial Arrangement of Output Volume



Layer Dimensions:

$h \times w \times d$

where h and w are spatial dimensions
 d (depth) = number of filters

Stride:

Filter step size

Receptive Field:

Locations in input image that
a node is path connected to



```
tf.keras.layers.Conv2D( filters=d, kernel_size=(h,w), strides=s )
```



```
torch.nn.Conv2d( in_channels=3, out_channels=d, kernel_size=(h,w), stride=s )
```

Need to specify
input dimensionality!

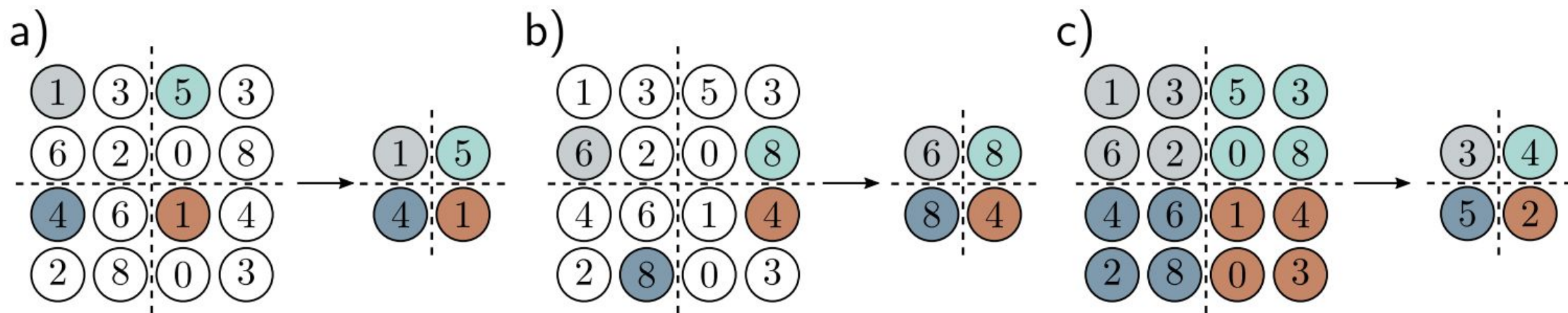
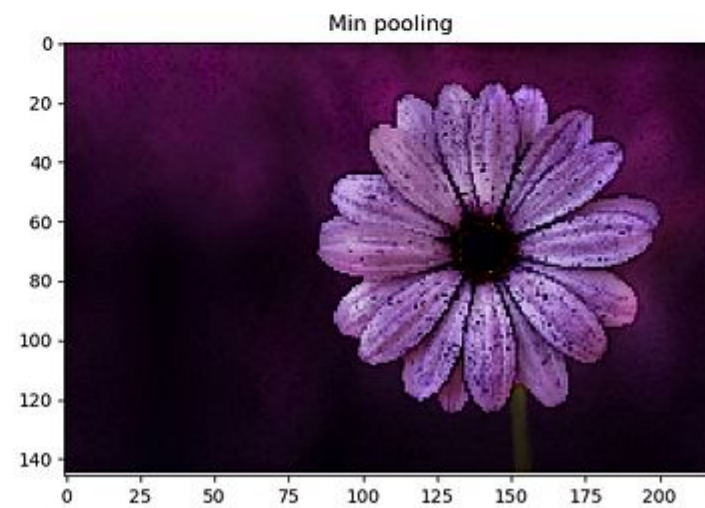
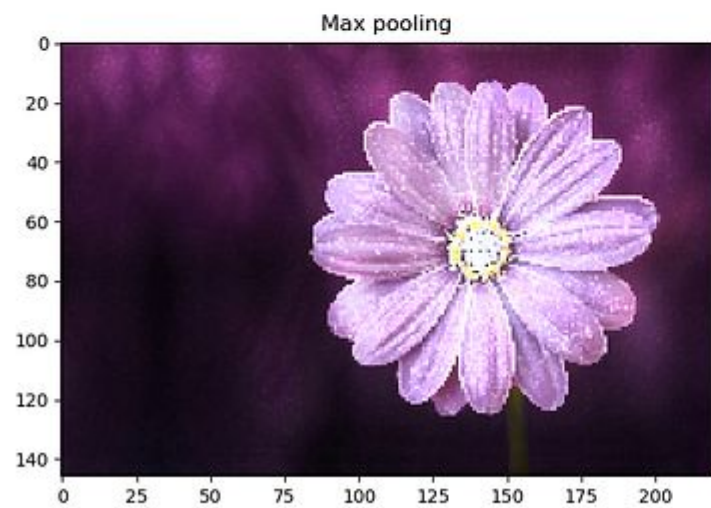
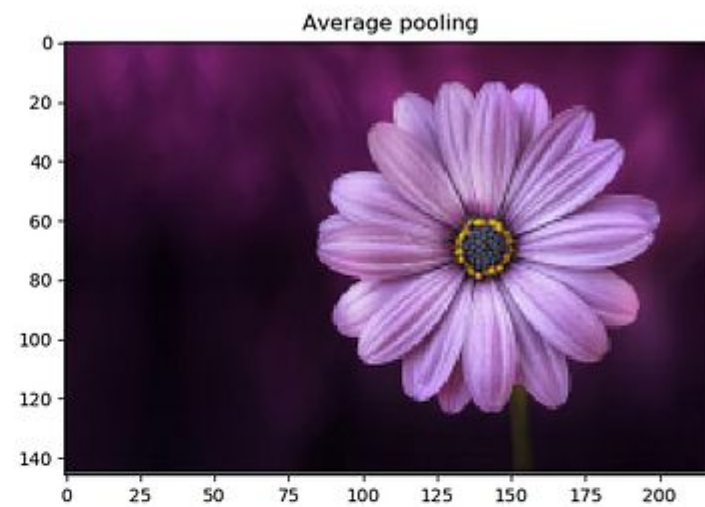


Figure 10.11 Methods for scaling down representation size (downsampling). a) Sub-sampling. The original 4×4 representation (left) is reduced to size 2×2 (right) by retaining every other input. Colors on the left indicate which inputs contribute to the outputs on the right. This is effectively what happens with a kernel of stride two, except that the intermediate values are never computed. b) Max pooling. Each output comprises the maximum value of the corresponding 2×2 block. c) Mean pooling. Each output is the mean of the values in the 2×2 block.



Receptive field

- Receptive field is defined as the size of the region in the input that produces the feature.
- In ANN, every hidden unit has access to whole input.
- In CNNs, the RF of units in successive layers increases.

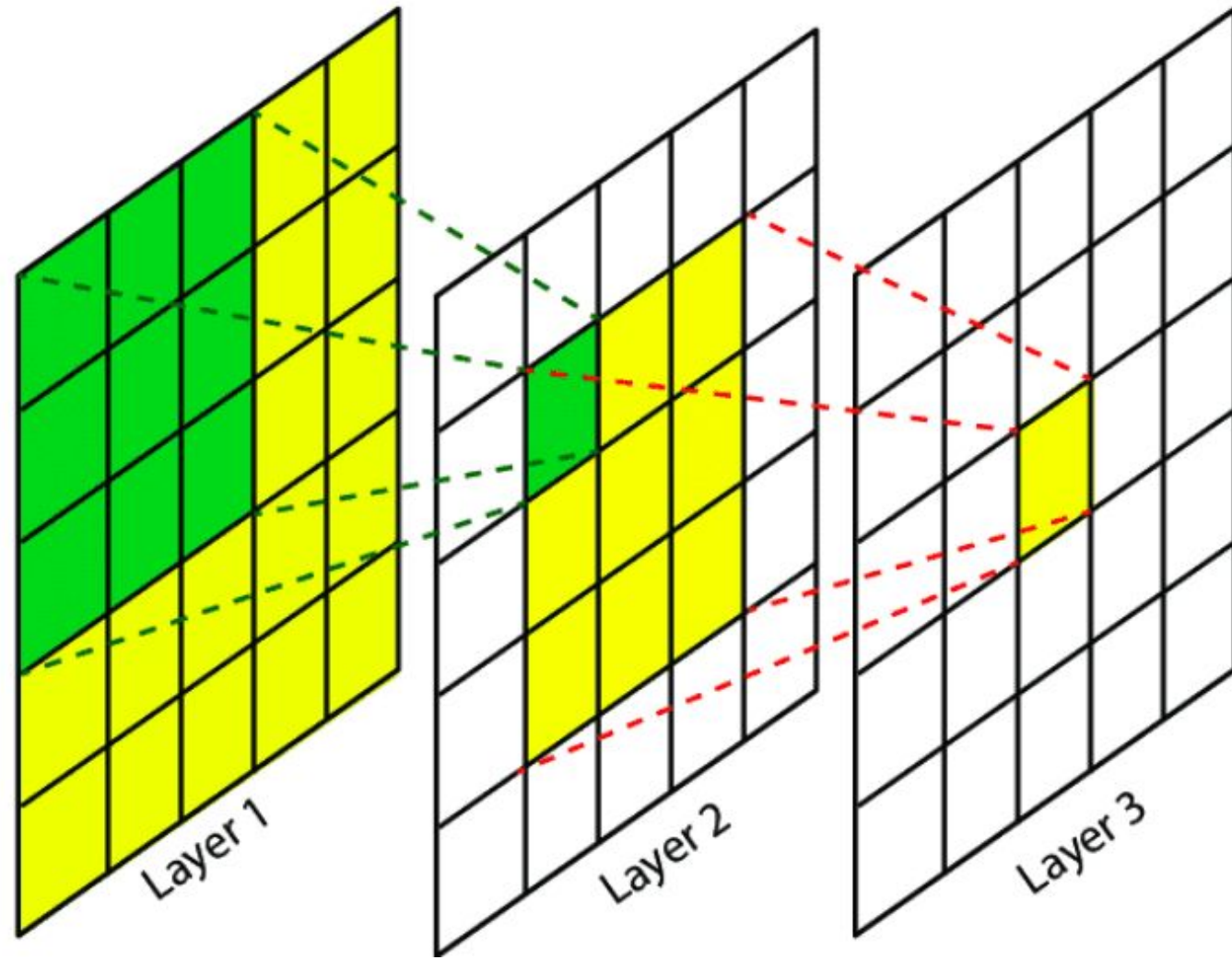
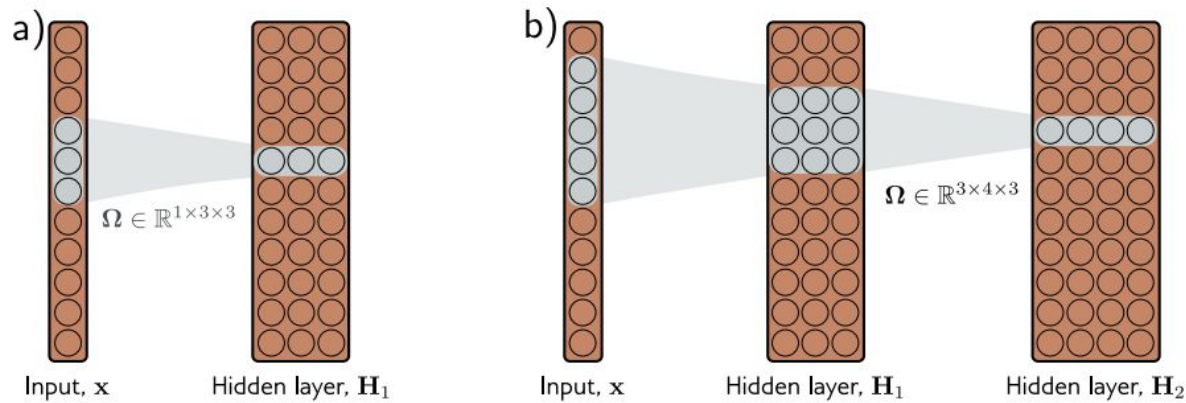
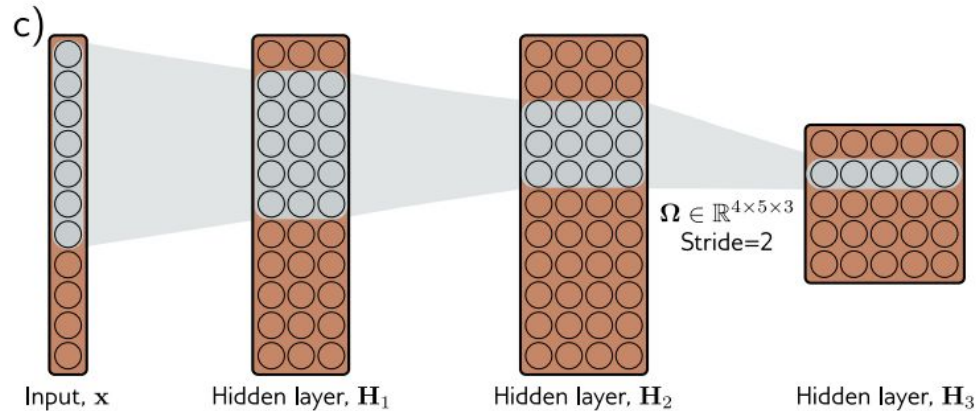


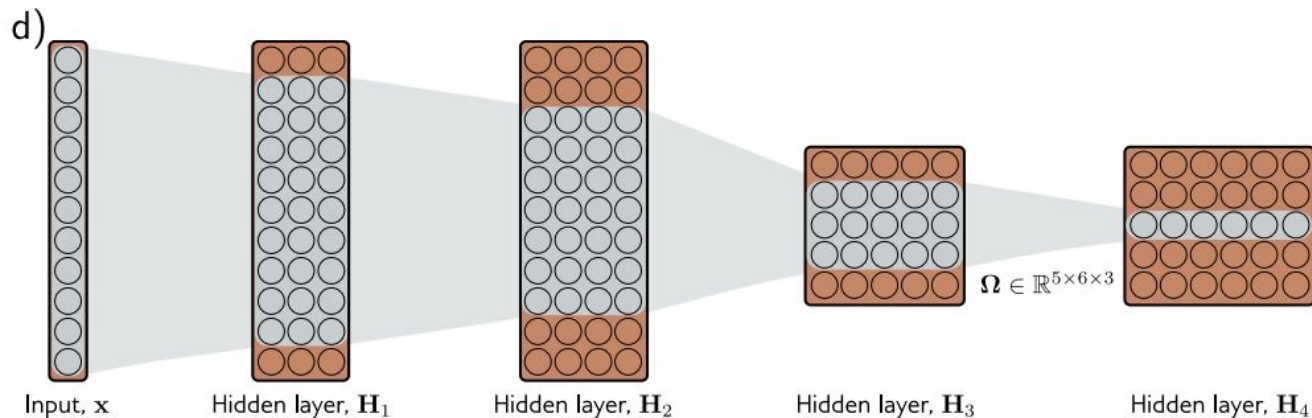
Figure 10.6 Receptive fields for network with kernel width of three.



a) An input with eleven dimensions feeds into a hidden layer with three channels and convolution kernel of size three. The pre-activations of the three highlighted hidden units in the first hidden layer H_1 are different weighted sums of the nearest three inputs, so the receptive field in H_1 has size three.



b) The pre-activations of the four highlighted hidden units in layer H_2 each take a weighted sum of the three channels in layer H_1 at each of the three nearest positions. Each hidden unit in layer H_1 weights the nearest three input positions. Hence, hidden units in H_2 have a receptive field size of five.



c) The hidden units in the third layer (kernel size three, stride two) increases the receptive field size to seven.

d) By the time we add a fourth layer, the receptive field of the hidden units at position three have a receptive field that covers the entire input.

Recap Quiz

1. Why do we prefer CNNs over standard fully connected networks for image processing?
 - a) CNNs are easier to code.
 - b) ANNs cannot handle color images
 - c) ANNs require too many weights and overfit
 - d) CNNs do not require GPU.

Quiz

2. If my input image is RGB (3 channels), my filter must also have depth of 3.
- a) True
 - b) False

Quiz

3. What is the primary benefit of Max pooling layer?
- a) It adds more pixels to the image.
 - b) It makes the image sharper.
 - c) It reduces dimensions and provides “Translation invariance”.
 - d) It converts the image to black and white.

Quiz

4. What happens if we perform convolution without Zero padding (Valid padding)?
- a) The input image gets larger.
 - b) We loose information at the border of the image.
 - c) The filter will crash.
 - d) The colors get inverted.

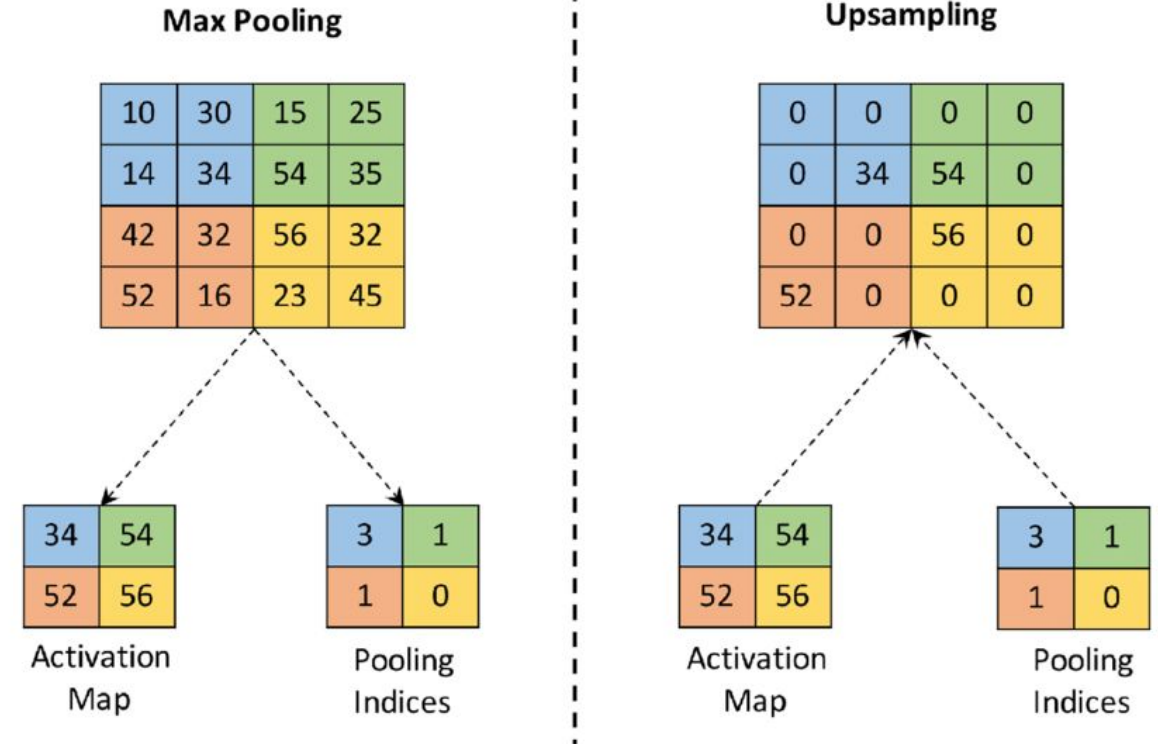
Upsampling: Transposed Convolution

Upsampling

Upsampling is a technique used to **increase the spatial resolution** of feature maps, effectively transforming low-resolution data into a higher-dimensional representation.

Common Methods:

- **Nearest Neighbor / Bilinear Interpolation:**
Simple, non-learnable techniques that fill new pixels based on surrounding values.
- **Transposed Convolution (Deconvolution):**
A learnable layer that uses weights to optimally reconstruct spatial detail during training.



Upsampling

- The simplest way to scale up a network layer to double the resolution is to duplicate all the channels at each spatial position four times.
- A second method is max unpooling; this is used where we have previously used a max pooling operation for downsampling, and we distribute the values to the positions they originated from.
- A third approach uses bilinear interpolation to fill in the missing values between the points where we have samples.
- A fourth approach is roughly analogous to downsampling using a stride of two. In that method, there were half as many outputs as inputs, and for kernel size three, each output was a weighted sum of the three closest inputs
- In transposed convolution, this picture is reversed. There are twice as many outputs as inputs, and each input contributes to three of the outputs. When we consider the associated weight matrix of this upsampling mechanism we see that it is the transpose of the matrix for the downsampling mechanism

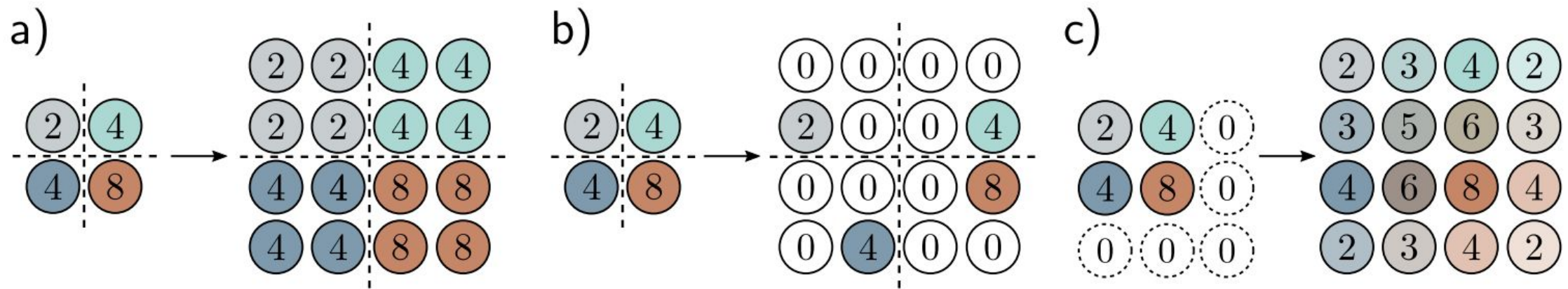


Figure 10.12 Methods for scaling up representation size (upsampling). a) The simplest way to double the size of a 2D layer is to duplicate each input four times. b) In networks where we have previously used a max pooling operation (figure 10.11b), we can redistribute the values to the same positions they originally came from (i.e., where the maxima were). This is known as max unpooling. c) A third option is bilinear interpolation between the input values.

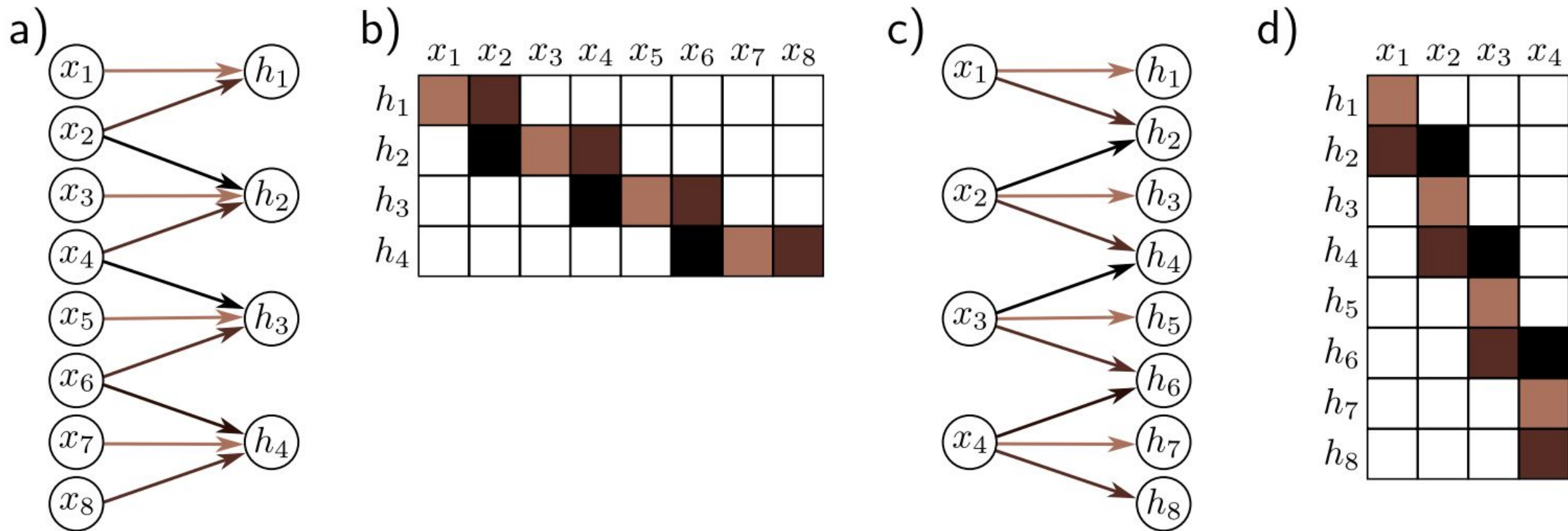
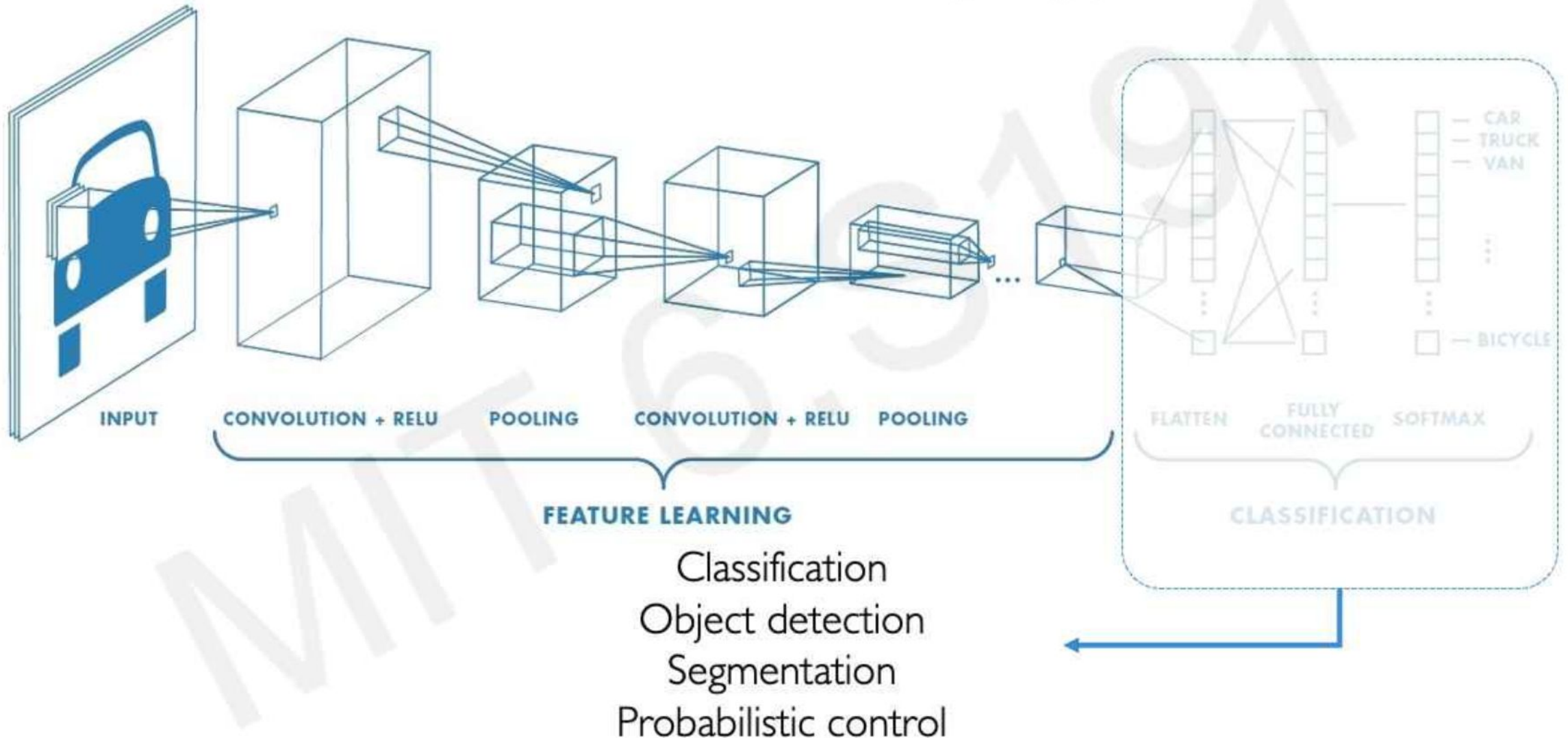


Figure 10.13 Transposed convolution in 1D. a) Downsampling with kernel size three, stride two, and zero-padding. Each output is a weighted sum of three inputs (arrows indicate weights). b) This can be expressed by a weight matrix (same color indicates shared weight). c) In transposed convolution, each input contributes three values to the output layer, which has twice as many outputs as inputs. d) The associated weight matrix is the transpose of that in panel (b).

An Architecture for Many Applications



Visualizing Standard Architectures

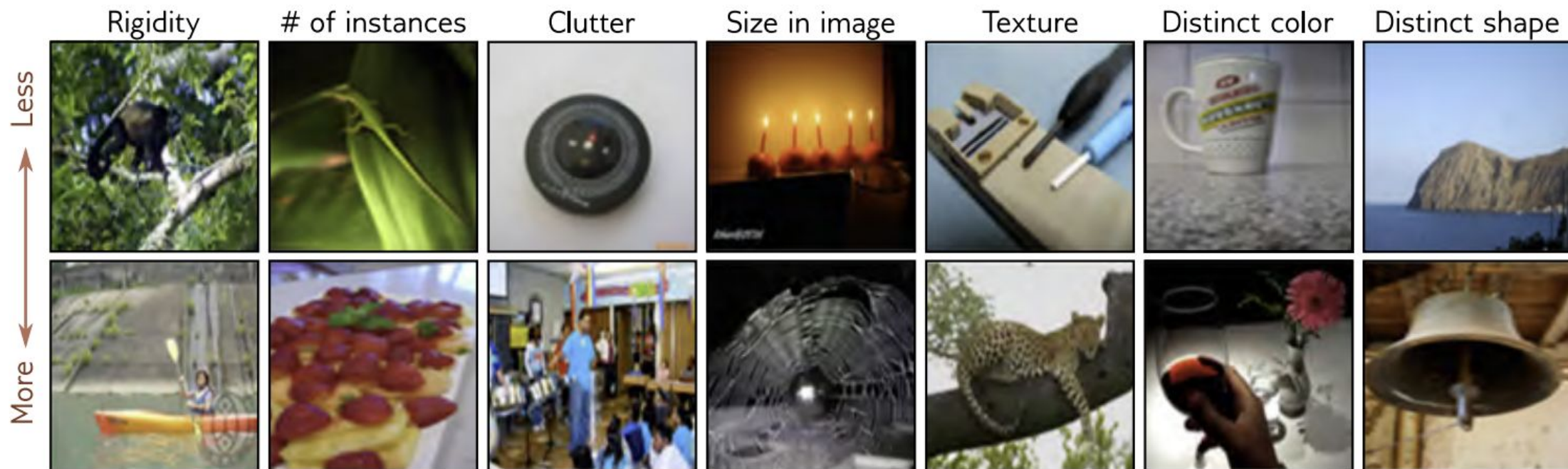
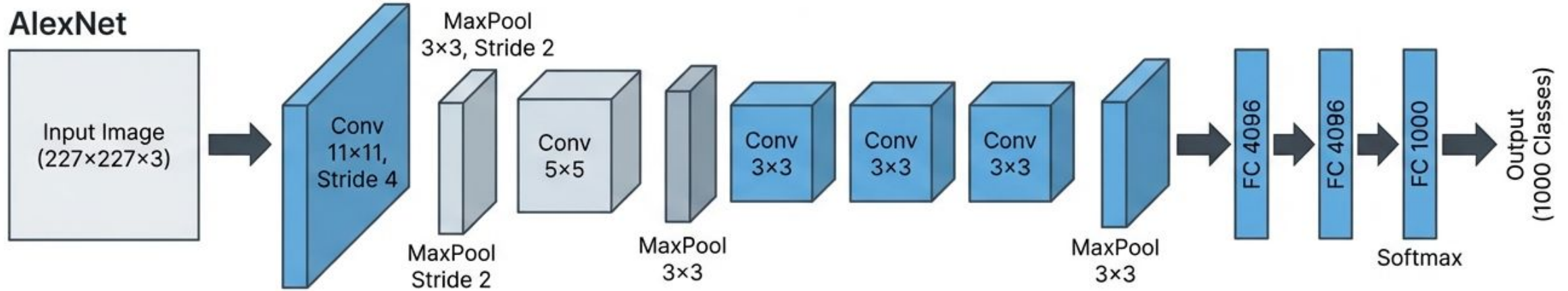


Figure 10.15 Example ImageNet classification images. The model aims to assign an input image to one of 1000 classes. This task is challenging because the images vary widely along different attributes (columns). These include rigidity (monkey < canoe), number of instances in image (lizard < strawberry), clutter (compass < steel drum), size (candle < spiderweb), texture (screwdriver < leopard), distinctiveness of color (mug < red wine), and distinctiveness of shape (headland < bell). Adapted from Russakovsky et al. (2015).

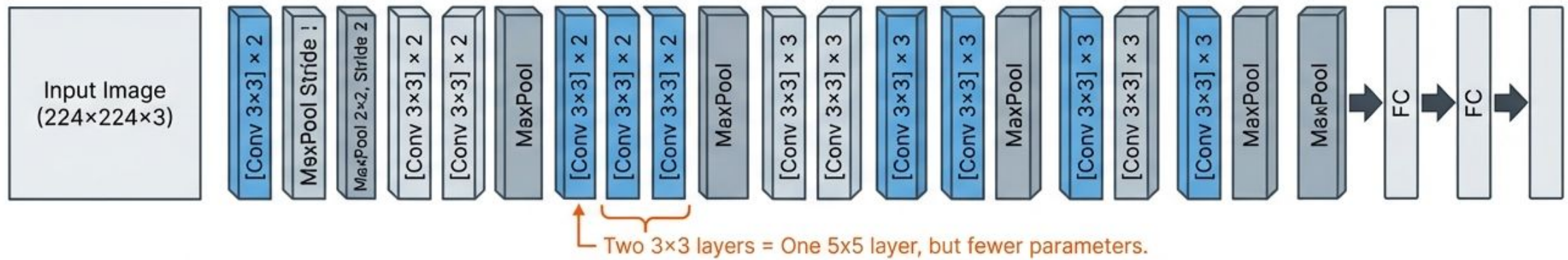
Application: Image Classification (AlexNet & VGG)

Trend: Spatial Size ↓, Channel Depth ↑

AlexNet



VGG



Alexnet (from Lecture 0)

In 2012, AlexNet was the first convolutional network to perform well on this task.

It consists of eight hidden layers with ReLU activation functions, of which the first five are convolutional and the rest fully connected..

This system achieved a 16.4% top-5 error rate and a 38.1% top-1 error rate. At the time, this was an **enormous leap forward** in performance at a task considered far beyond the capabilities of contemporary methods.

This result revealed the potential of deep learning and kick-started the modern era of AI research

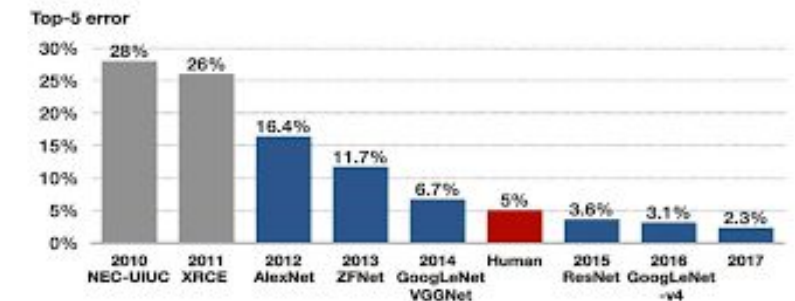
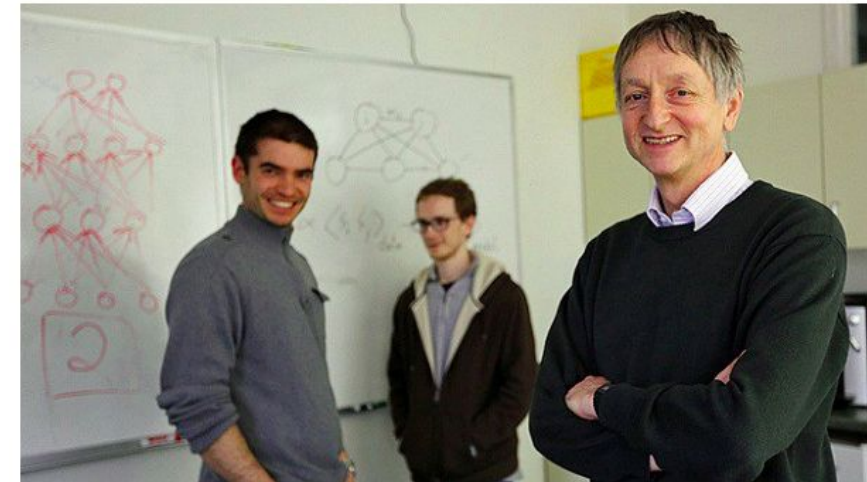
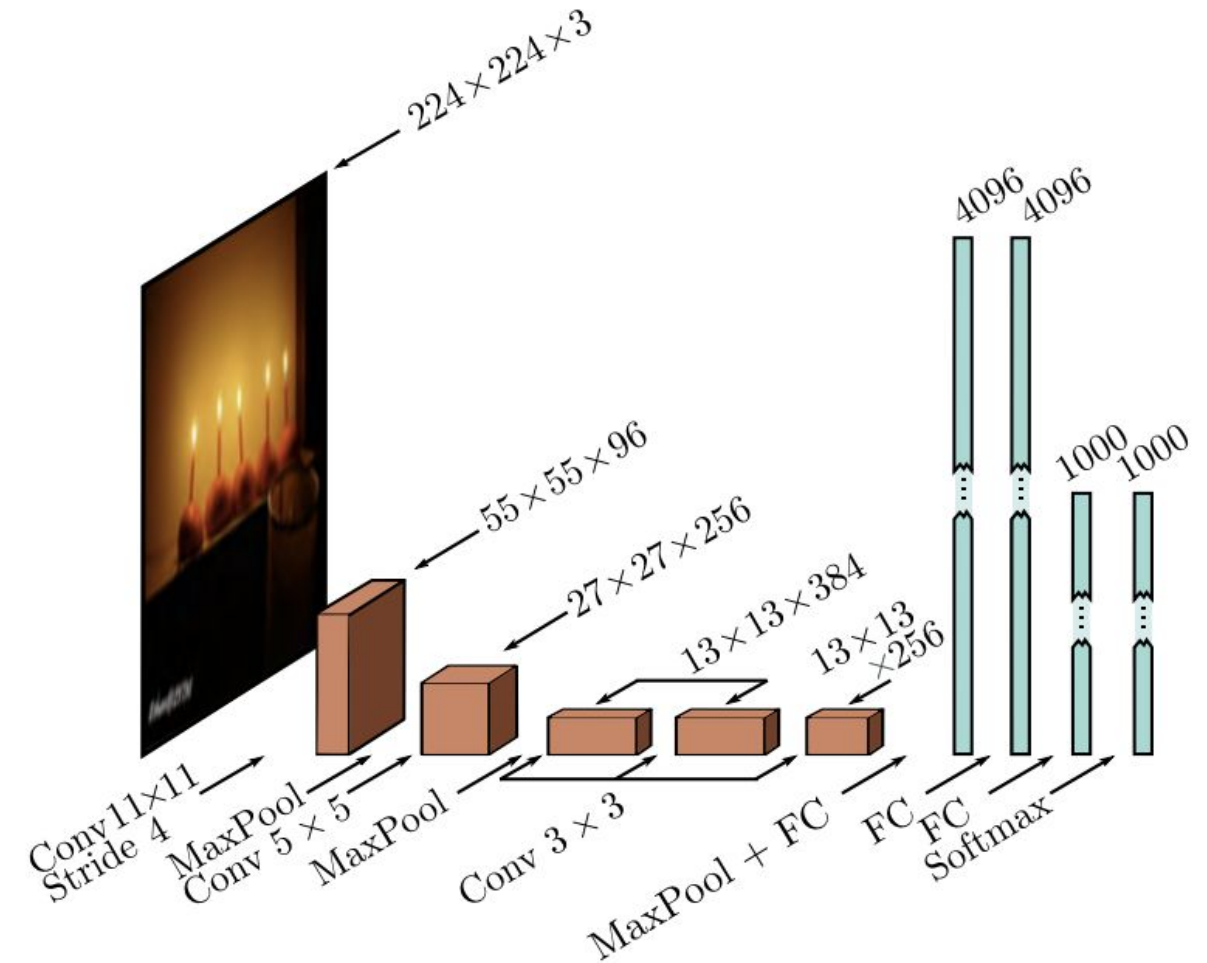


Figure 10.16 AlexNet (Krizhevsky et al., 2012). The network maps a 224×224 color image to a 1000-dimensional vector representing class probabilities. The network first convolves with 11×11 kernels and stride 4 to create 96 channels. It decreases the resolution again using a max pool operation and applies a 5×5 convolutional layer. Another max pooling layer follows, and three 3×3 convolutional layers are applied. After a final max pooling operation, the result is vectorized and passed through three fully connected (FC) layers and finally the softmax layer.



VGG network (Visual Geometry Group)

The VGG network was also targeted at classification in the ImageNet task and achieved a considerably better performance of 6.8% top-5 error rate and a 23.7% top-1 error rate.

The VGG network was also trained using data augmentation, weight decay, and dropout.



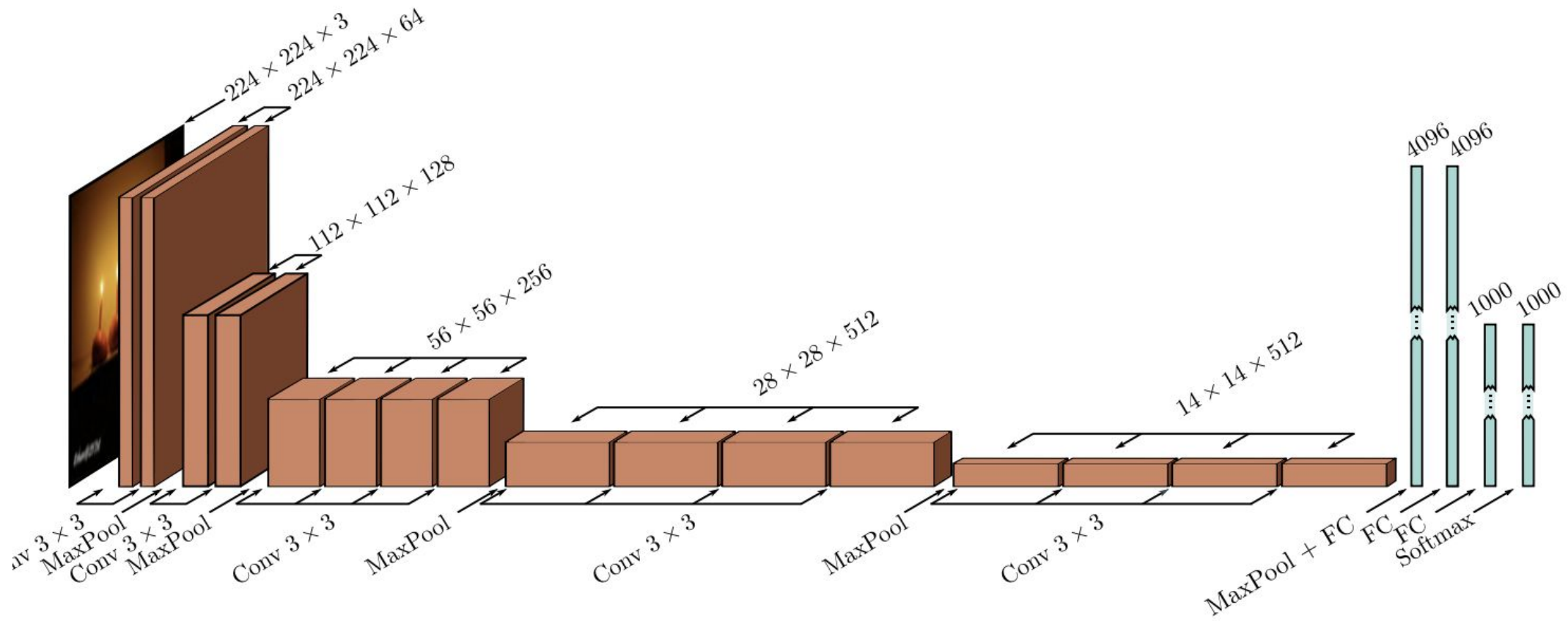


Figure 10.17 VGG network (Simonyan & Zisserman, 2014) depicted at the same scale as AlexNet (see figure 10.16). This network consists of a series of convolutional layers and max pooling operations, in which the spatial scale of the representation gradually decreases, but the number of channels gradually increases. The hidden layer after the last convolutional operation is resized to a 1D vector and three fully connected layers follow. The network outputs 1000 activations corresponding to the class labels that are passed through a softmax function to create class probabilities.

Diff b/w Alexnet and VGG

The most important change between AlexNet and VGG was the depth of the network. The latter used 19 hidden layers and 144 million parameters. The networks are depicted at the same scale for comparison.

There was a general trend for several years for performance on this task to improve as the depth of the networks increased, and this is evidence that depth is important in neural networks.

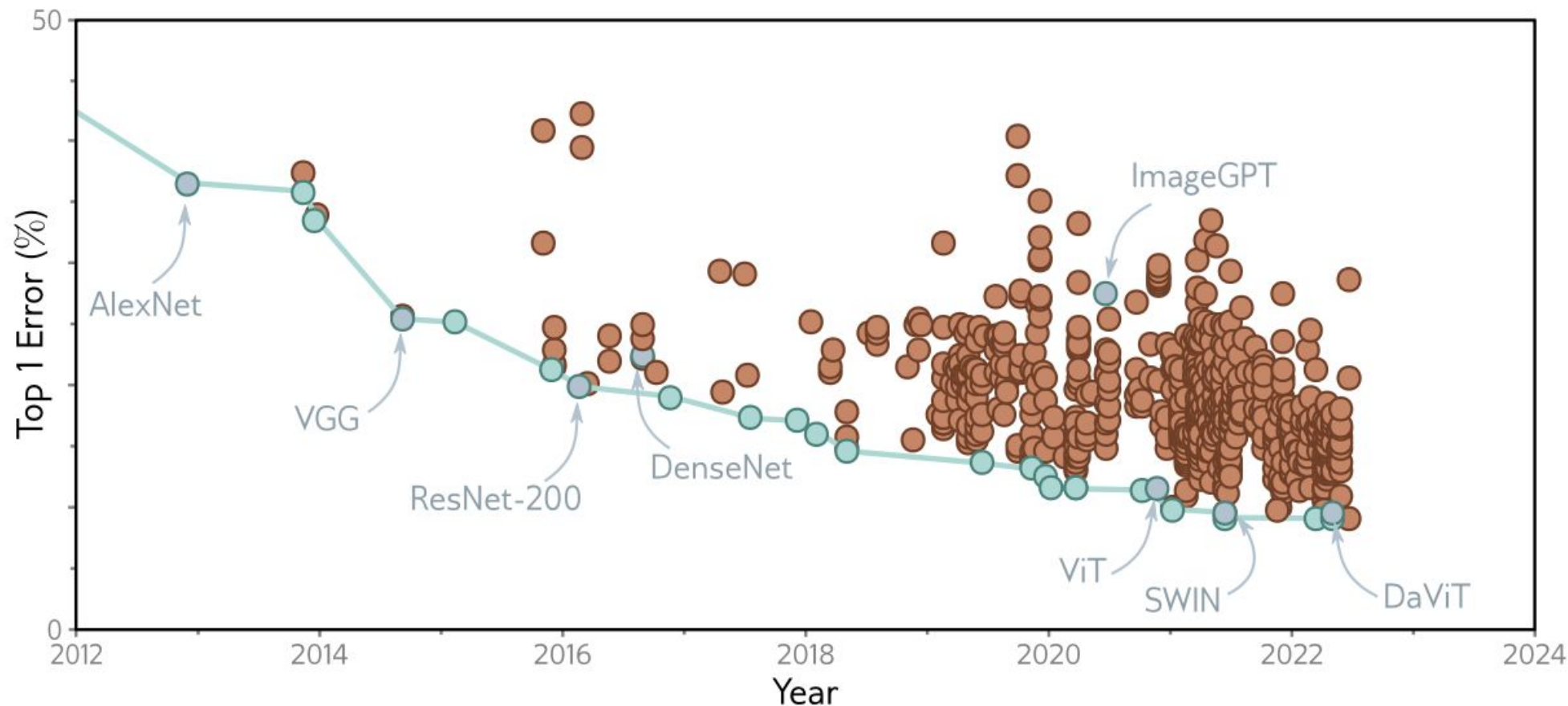


Figure 10.21 ImageNet performance. Each circle represents a different published model. Blue circles represent models that were state-of-the-art. Models discussed in this book are also highlighted. The AlexNet and VGG networks were remarkable for their time but are now far from state of the art. ResNet-200 and DenseNet are discussed in chapter 11. ImageGPT, ViT, SWIN, and DaViT are discussed in chapter 12. Adapted from <https://paperswithcode.com/sota/image-classification-on-imagenet>.

Every network we have seen so far processes the data sequentially; each layer receives the previous layer's output and passes the result to the next

$$\mathbf{y} = \mathbf{f}_4 \left[\mathbf{f}_3 \left[\mathbf{f}_2 \left[\mathbf{f}_1 [\mathbf{x}, \phi_1], \phi_2 \right], \phi_3 \right], \phi_4 \right]$$

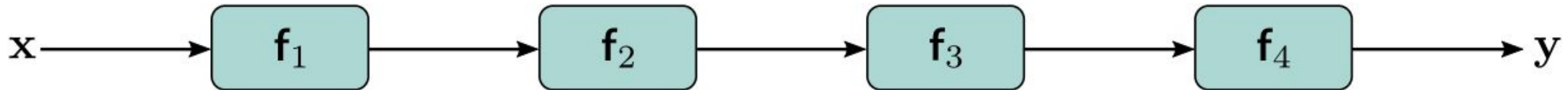


Figure 11.1 Sequential processing. Standard neural networks pass the output of each layer directly into the next layer.

Limitations of sequential processing

In principle, we can add as many layers as we want, and we saw that adding more layers to a convolutional network does improve performance; the VGG network outperforms AlexNet as its deeper.

However, image classification performance decreases again as further layers are added. This is surprising since models generally perform better as more capacity is added.

Indeed, the decrease is present for both the training set and the test set, which implies that the problem is **training** deeper networks rather than the inability of deeper networks to generalize.

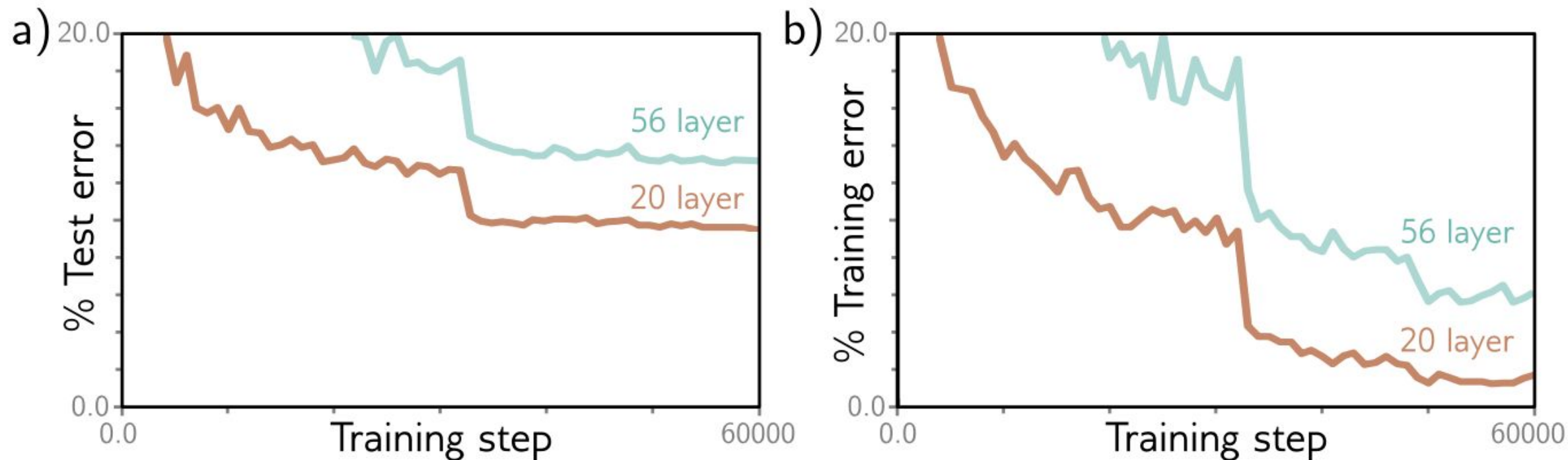


Figure 11.2 Decrease in performance when adding more convolutional layers. a) A 20-layer convolutional network outperforms a 56-layer neural network for image classification on the test set of the CIFAR-10 dataset (Krizhevsky & Hinton, 2009). b) This is also true for the training set, which suggests that the problem relates to training the original network rather than a failure to generalize to new data. Adapted from He et al. (2016a).

The Cause: Shattered Gradients

In deep sequential networks, the correlation between input and gradient drops to zero.

Shallow Network (Smooth Gradient)



Gradient is stable and correlated.

Deep Network (Shattered Gradient)



Gradient behaves like white noise.
Steps are random.

Metaphor: The Mountain Range.

Shallow Net = Smooth Hill. Deep Net = Chaotic range of tiny peaks. A small step moves you to a completely unrelated gradient.

Shattered Gradient Problem

Any reasonable choice of step size may move to a place with a completely different and unrelated gradient; the loss surface looks like an enormous range of tiny mountains rather than a single smooth structure that is easy to descend.

For a shallow network, the gradient of the output with respect to the input changes slowly as we change the input.

However, for a deep network, a tiny change in the input results in a completely different gradient. This is captured by the autocorrelation function of the gradient.

Nearby gradients are correlated for shallow networks, but this correlation quickly drops to zero for deep networks

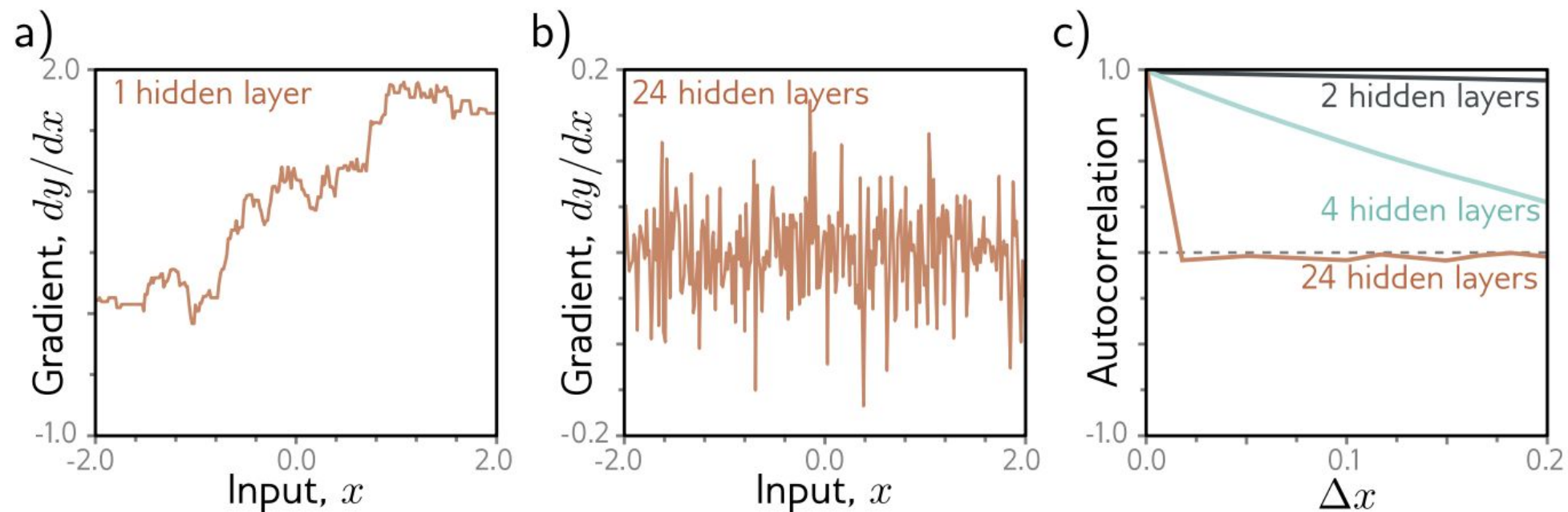


Figure 11.3 Shattered gradients. a) Consider a shallow network with 200 hidden units and Glorot initialization (He initialization without the factor of two) for both the weights and biases. The gradient $\partial y / \partial x$ of the scalar network output y with respect to the scalar input x changes relatively slowly as we change the input x . b) For a deep network with 24 layers and 200 hidden units per layer, this gradient changes very quickly and unpredictably. c) The autocorrelation function of the gradient shows that nearby gradients become unrelated (have autocorrelation close to zero) for deep networks. This *shattered gradients* phenomenon may explain why it is hard to train deep networks. Gradient descent algorithms rely on the loss surface being relatively smooth, so the gradients should be related before and after each update step. Adapted from Balduzzi et al. (2017).

Residual Blocks

Residual (skip) connection

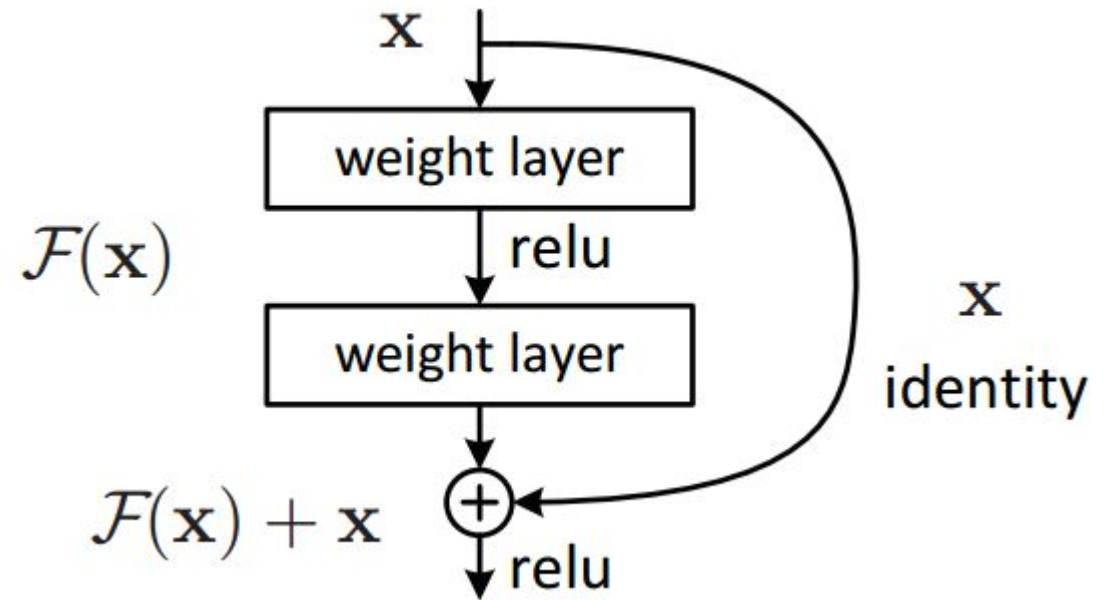
Residual or skip connections are branches in the computational path, whereby the input to each network layer $f[\bullet]$ is added back to the output.

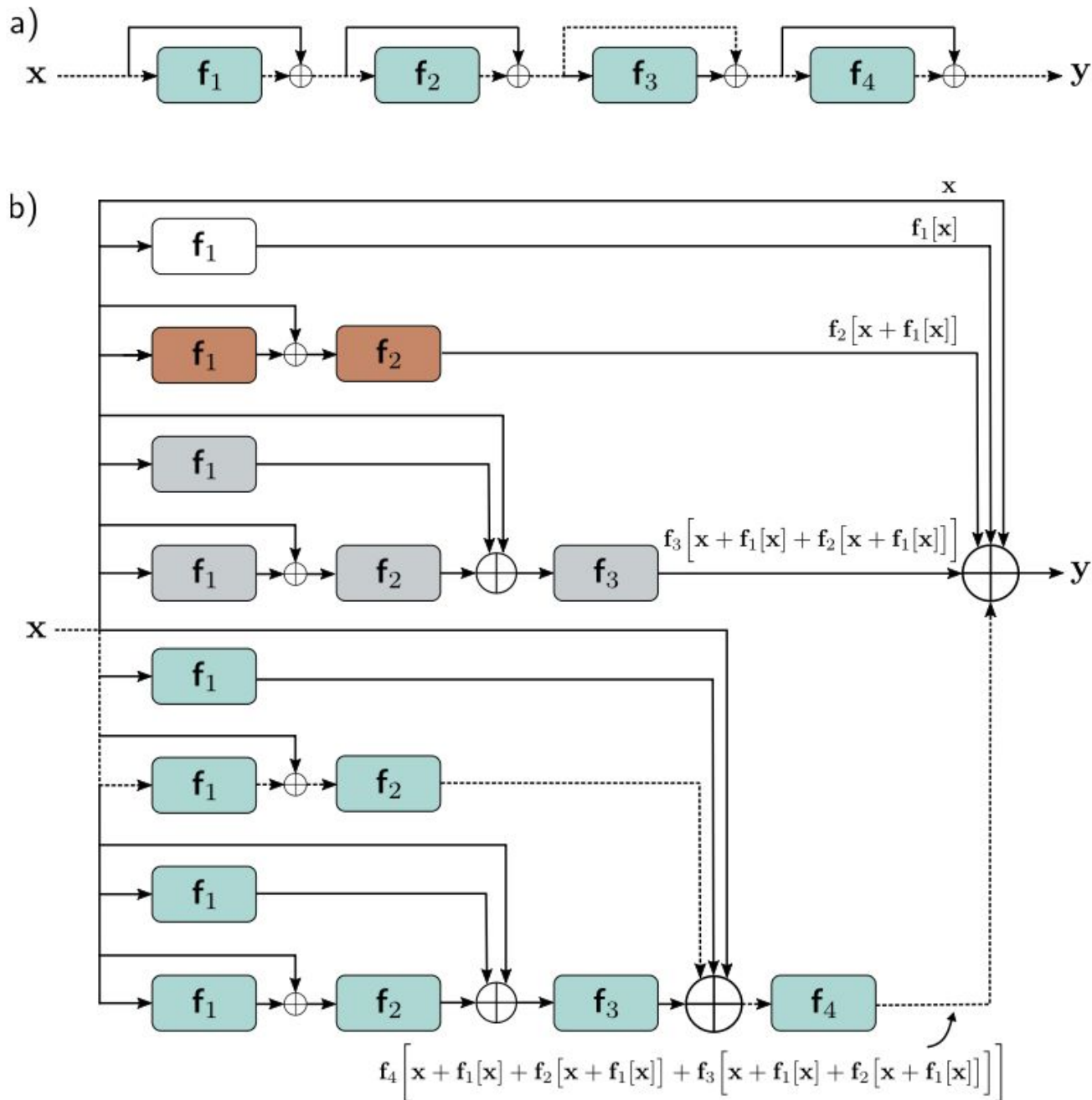
The residual network is defined as:

$$\begin{aligned} \mathbf{h}_1 &= \mathbf{x} + \mathbf{f}_1[\mathbf{x}, \phi_1] \\ \mathbf{h}_2 &= \mathbf{h}_1 + \mathbf{f}_2[\mathbf{h}_1, \phi_2] \\ \mathbf{h}_3 &= \mathbf{h}_2 + \mathbf{f}_3[\mathbf{h}_2, \phi_3] \\ \mathbf{y} &= \mathbf{h}_3 + \mathbf{f}_4[\mathbf{h}_3, \phi_4], \end{aligned}$$

Thereby making the overall function as:

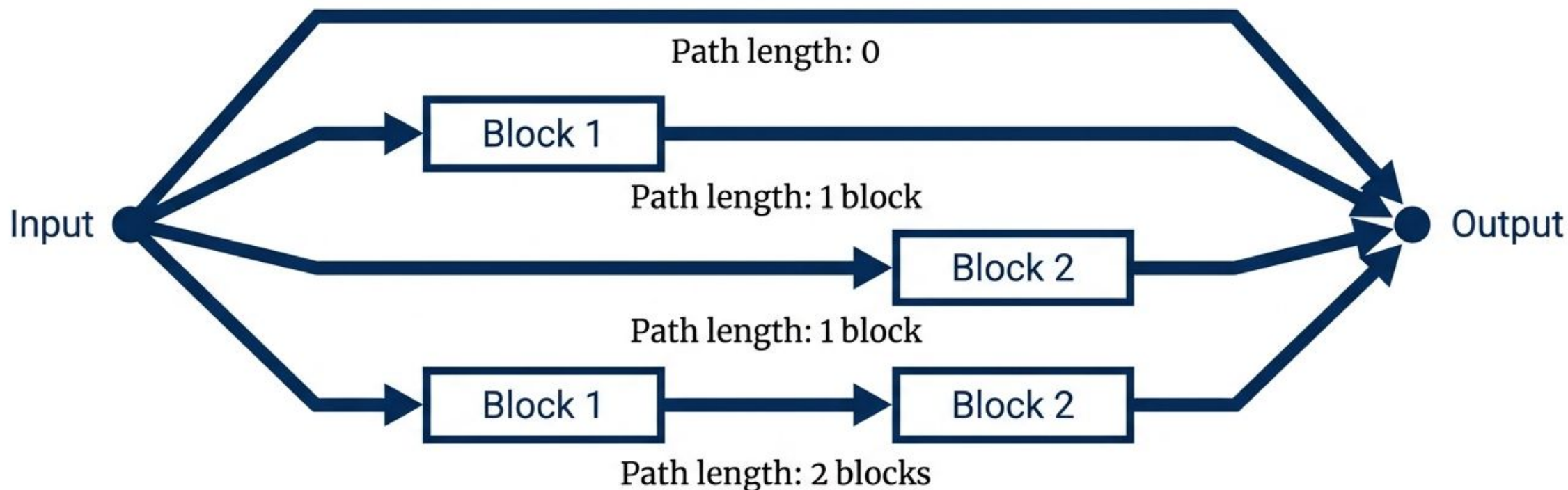
$$\begin{aligned} \mathbf{y} = & \mathbf{x} + \mathbf{f}_1[\mathbf{x}] \\ & + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]] \\ & + \mathbf{f}_3[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]]] \\ & + \mathbf{f}_4\left[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]] + \mathbf{f}_3[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]]]\right], \end{aligned}$$





Intuition: The Network as an Ensemble

ResNets behave like an ensemble of shallower networks.



In a 54-block network, most gradient updates come from paths only 5–17 blocks long (Veit et al., 2016).

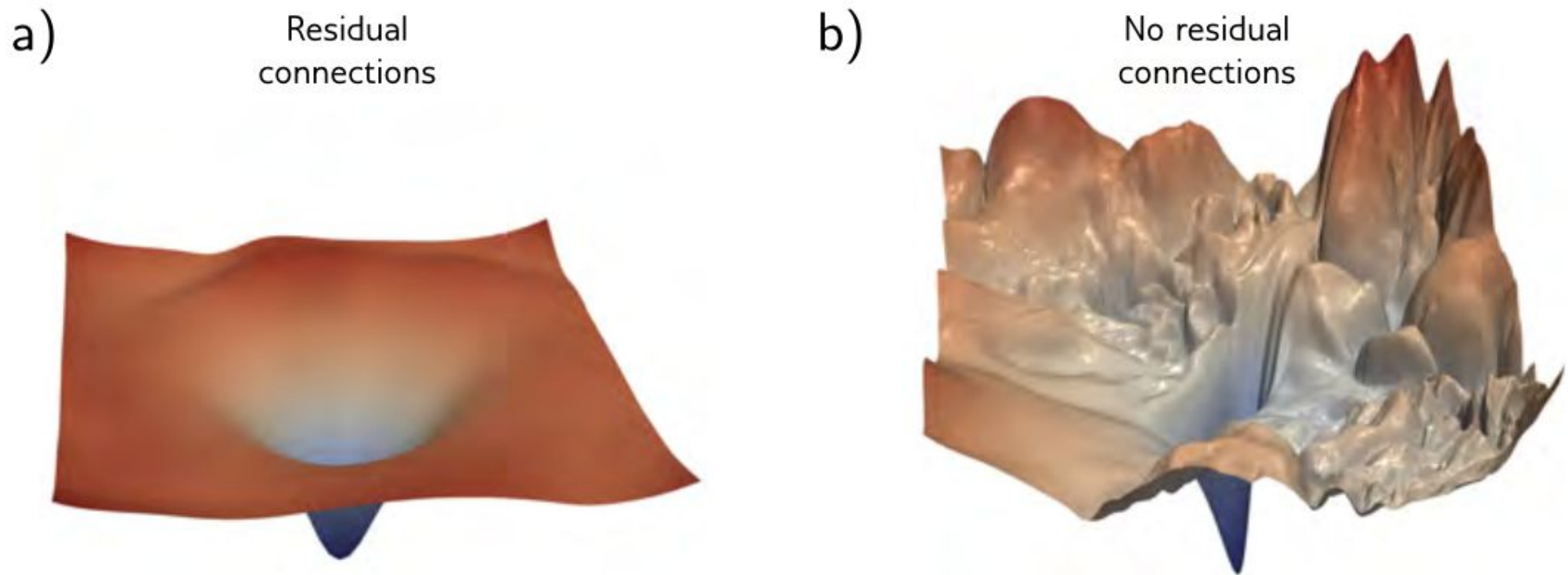
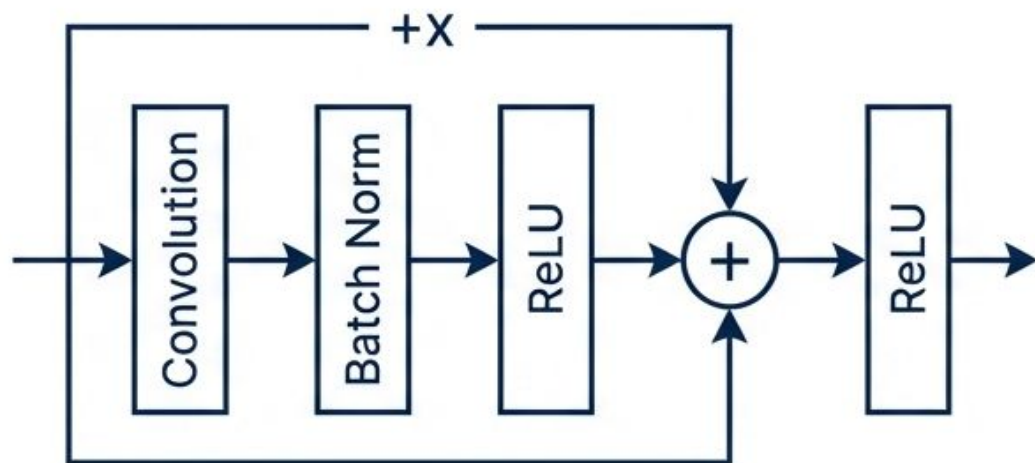


Figure 11.13 Visualizing neural network loss surfaces. Each plot shows the loss surface in two random directions in parameter space around the minimum found by SGD for an image classification task on the CIFAR-10 dataset. These directions are normalized to facilitate side-by-side comparison. a) Residual net with 56 layers. b) Results from the same network without skip connections. The surface is smoother with the skip connections. This facilitates learning and makes the final network performance more robust to minor errors in the parameters, so it will likely generalize better. Adapted from Li et al. (2018b).

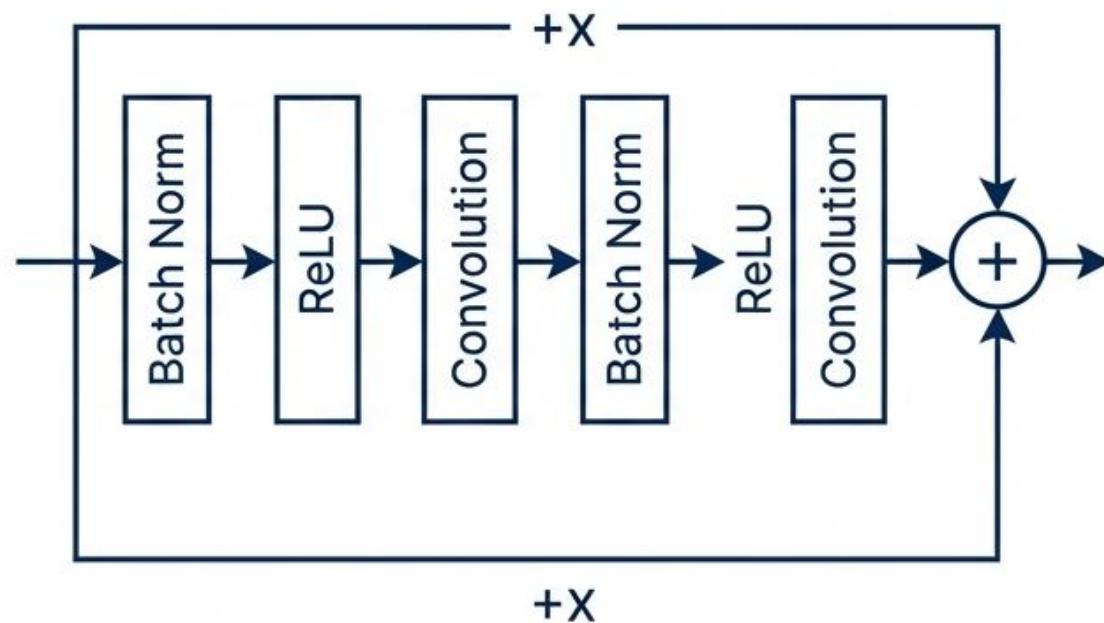
Implementation Detail: Order of Operations

Original (Post-activation)



Output is always non-negative (ReLU at end).

ResNet v2 (Pre-activation)



Output can be negative. Identity path is clean.

Pre-activation blocks allow the residual path to remain a pure identity mapping, improving signal propagation.

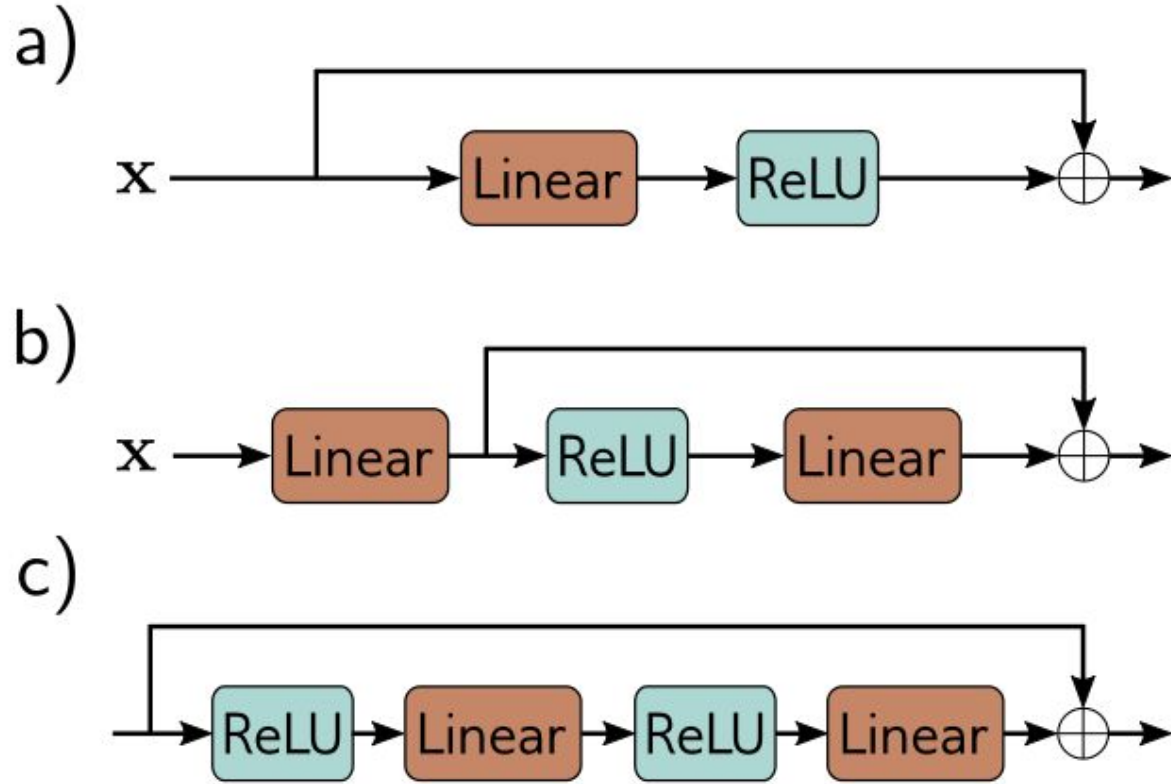
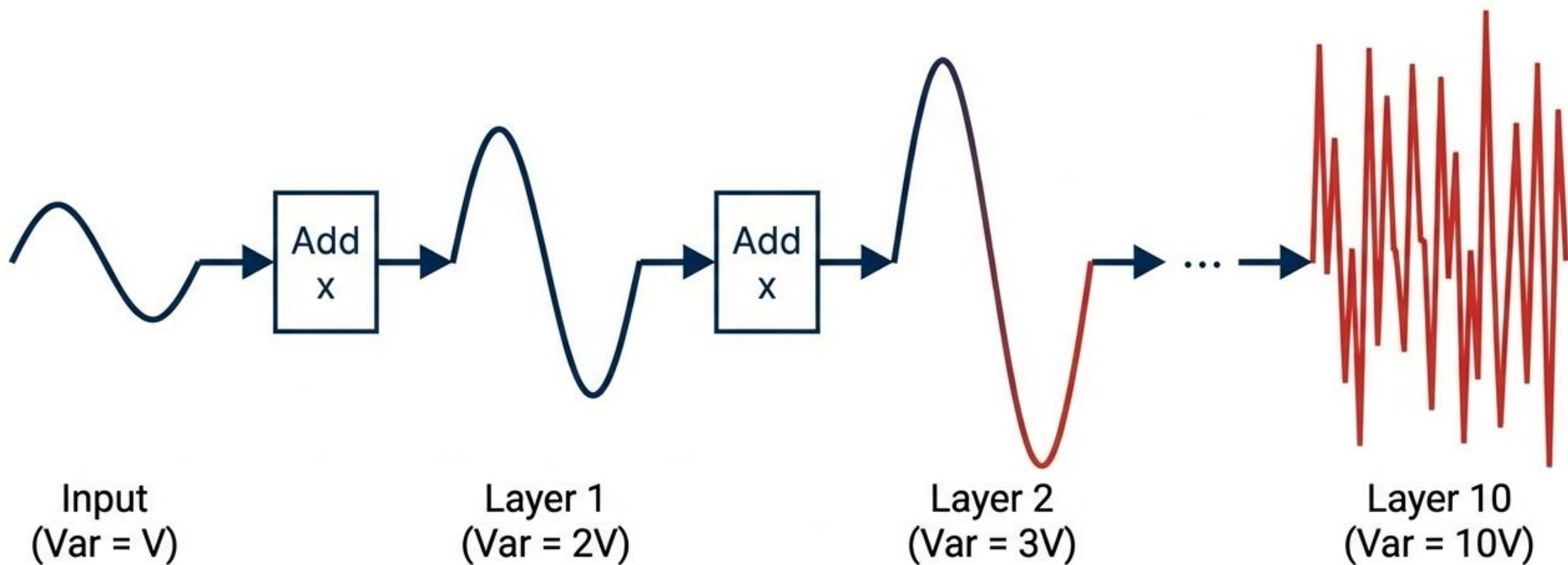


Figure 11.5 Order of operations in residual blocks. a) The usual order of linear transformation or convolution followed by a ReLU nonlinearity means that each residual block can only add non-negative quantities. b) With the reverse order, both positive and negative quantities can be added. However, we must add a linear transformation at the start of the network in case the input is all negative. c) In practice, it's common for a residual block to contain several network layers.

The New Challenge: Exploding Variance

Variance out = Variance in + Variance of block. Signal doubles at every step.



Consequence: Values exceed floating-point precision or cause instability.

Exploding variance problem

We initialize the network parameters so that the expected variance of the activations and gradients remains the same between layers.

He initialization achieves this for ReLU activations.

But even if we use He initialization within the residual block, the values in the forward pass increase exponentially.

Consider that we add the result of the processing in the residual block back to the input. Each branch has some (uncorrelated) variability. The overall variance increases when we recombine them. With ReLU activations and He initialization, the expected variance is unchanged by the processing in each block.

When we recombine with the input, the variance doubles, growing exponentially with the number of residual blocks.

Batch Normalization

Batch Normalization

BatchNorm shifts and rescales each activation h so that its mean and variance across the batch B become values that are learned during training.

$$m_h = \frac{1}{|B|} \sum_{i \in B} h_i$$
$$s_h = \sqrt{\frac{1}{|B|} \sum_{i \in B} (h_i - m_h)^2},$$

Then we use these statistics to standardize the batch activations to have mean zero and unit variance.

Finally, the normalized variable is scaled by γ and shifted by δ : $h_i \leftarrow \gamma h_i + \delta$

After this operation, the activations have mean δ and standard deviation γ across all members of the batch. Both of these quantities are learned during training

The Stabilizer: Batch Normalization

Re-centering and re-scaling activations at every step.

Step 1: Compute Moments

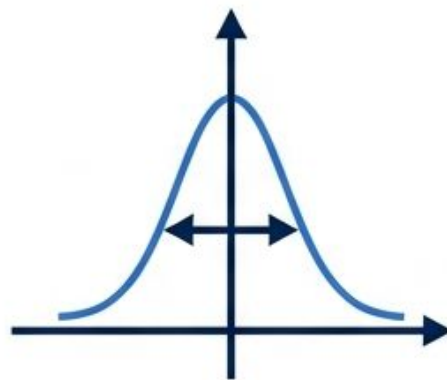
$$\text{Mean: } m_h = \frac{1}{|B|} \sum_i^n h_i$$

$$\text{Variance: } s_h = \sqrt{\frac{1}{|B|} \sum_i^n (h_i - m_h)^2}$$

Step 2: Normalize

Subtract mean, divide by deviation.

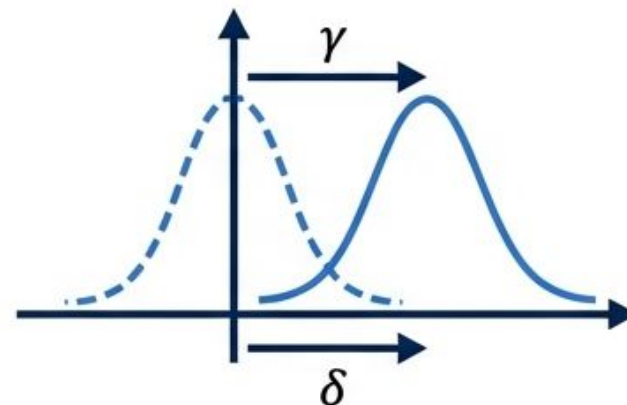
$$h_{new} = \frac{h - m_h}{s_h + \epsilon}$$



Step 3: Scale & Shift

Restore representational power with learnable parameters.

$$y = \gamma * h_{new} + \delta$$



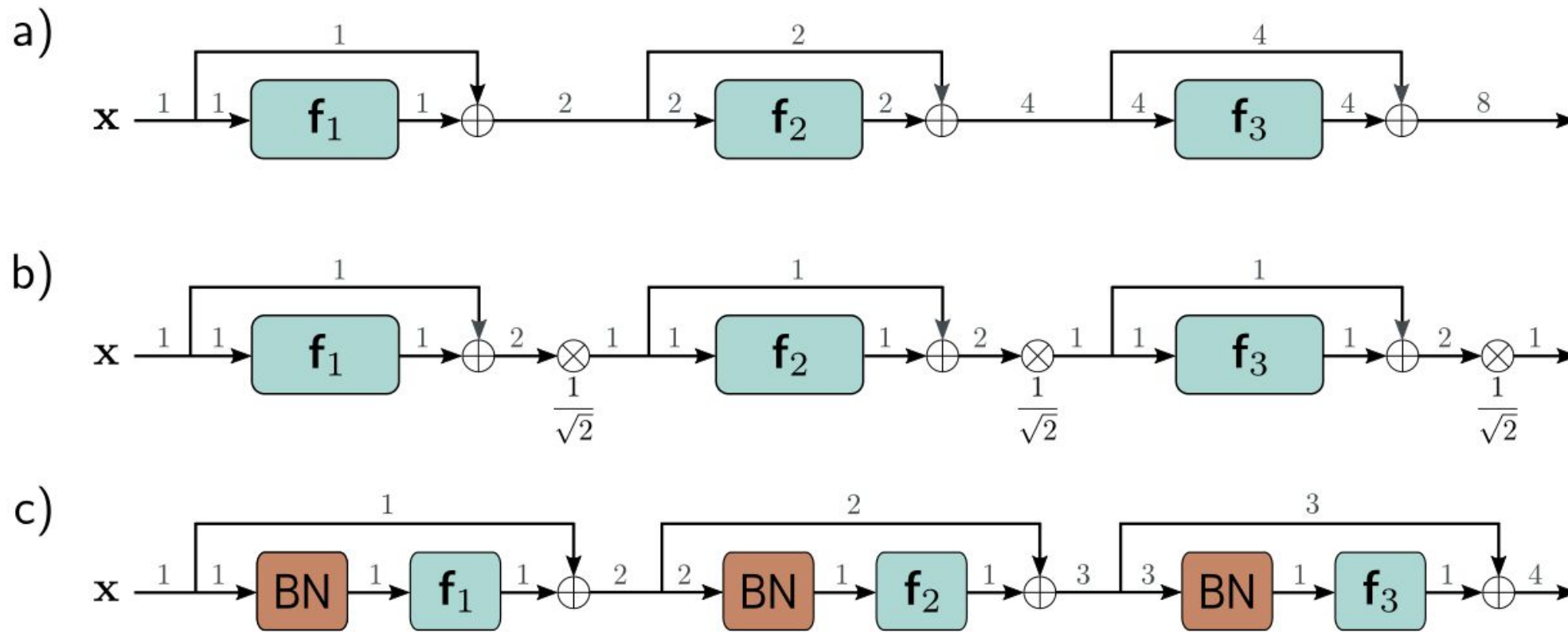


Figure 11.6 Variance in residual networks.

a) He initialization ensures that the expected variance remains unchanged after a linear plus ReLU layer f_k . Unfortunately, in residual networks, the input of each block is added back to the output, so the variance doubles at each layer (gray numbers indicate variance) and grows exponentially.

b) One approach would be to rescale the signal by $1/\sqrt{2}$ between each residual block.

c) A second method uses batch normalization (BN) as the first step in the residual block and initializes the associated offset δ to zero and scale γ to one. This transforms the input to each layer to have unit variance, and with He initialization, the output variance will also be one. Now the variance increases linearly with the number of residual blocks. A side-effect is that, at initialization, later network layers are dominated by the residual connection and are hence close to computing the identity.

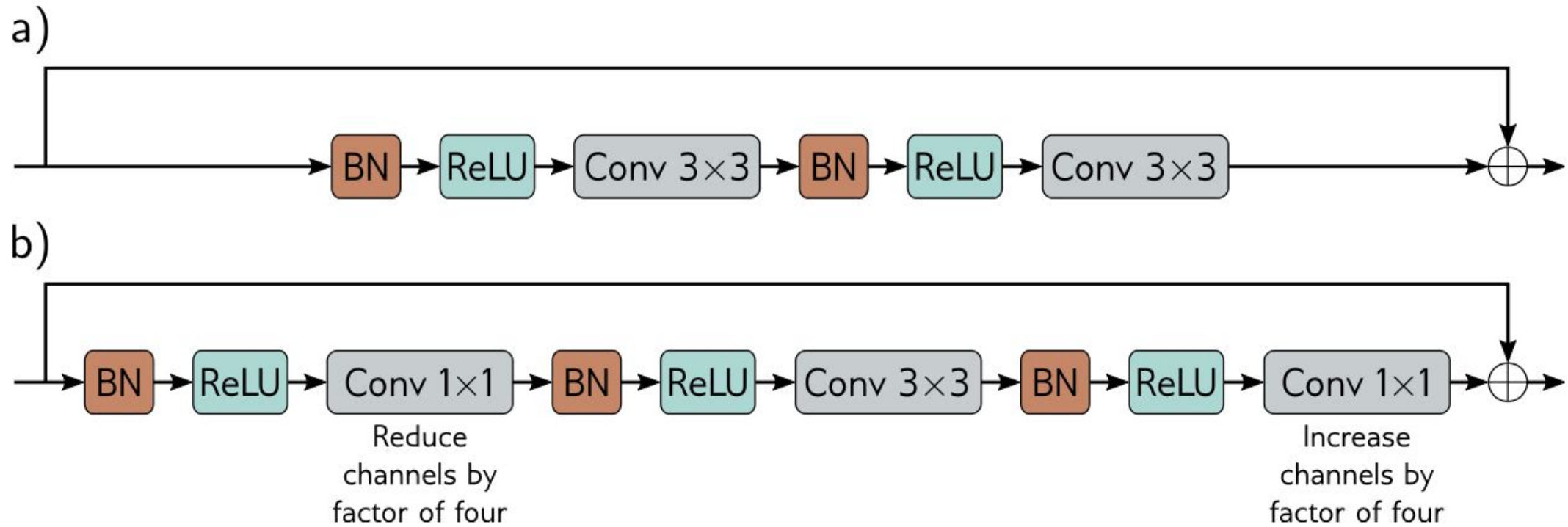


Figure 11.7 ResNet blocks. a) A standard block in the ResNet architecture contains a batch normalization operation, followed by an activation function, and a 3×3 convolutional layer. Then, this sequence is repeated. b). A bottleneck ResNet block still integrates information over a 3×3 region but uses fewer parameters. It contains three convolutions. The first 1×1 convolution reduces the number of channels. The second 3×3 convolution is applied to the smaller representation. A final 1×1 convolution increases the number of channels again so that it can be added back to the input.

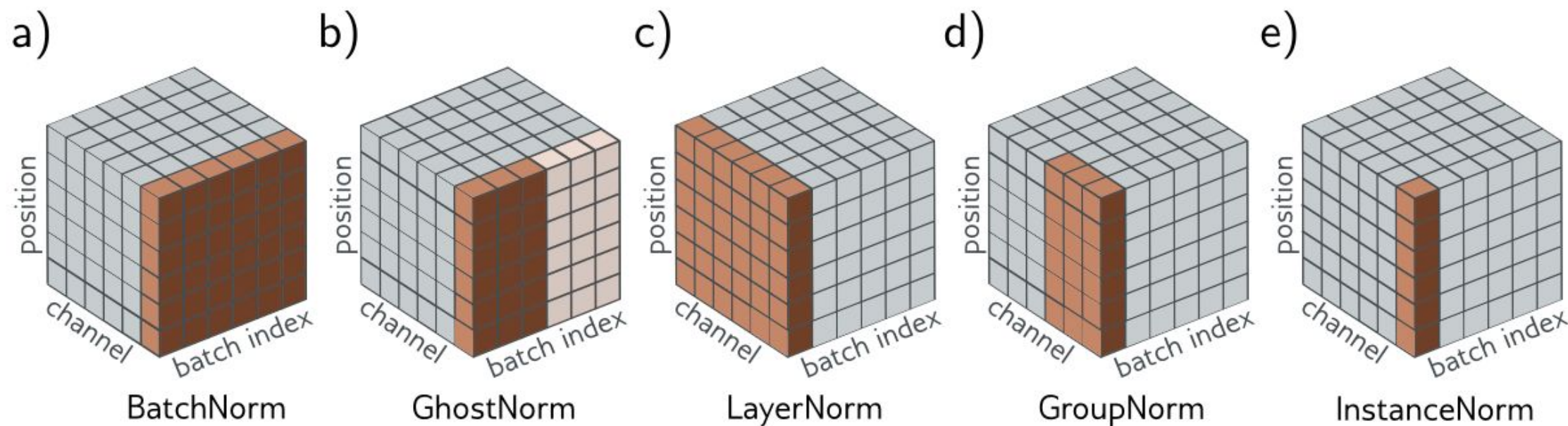


Figure 11.14 Normalization schemes. BatchNorm modifies each channel separately but adjusts each batch member in the same way based on statistics gathered across the batch and spatial position. Ghost batch normalization computes these statistics from only part of the batch to make them more variable. LayerNorm computes statistics for each batch member separately, based on statistics gathered across the channels and spatial position. It retains a separate learned scaling factor for each channel. GroupNorm normalizes within each group of channels and also retains a separate scale and offset parameter for each channel. InstanceNorm normalizes within each channel separately, computing the statistics only across spatial position. Adapted from Wu & He (2018).

Batch Norm: Costs & Benefits



Costs

Computation: Requires calculating mean/variance for every batch.

Memory: Must store intermediate stats during training.

Inference Complexity: Must use frozen global statistics (running averages) instead of batch stats at test time.

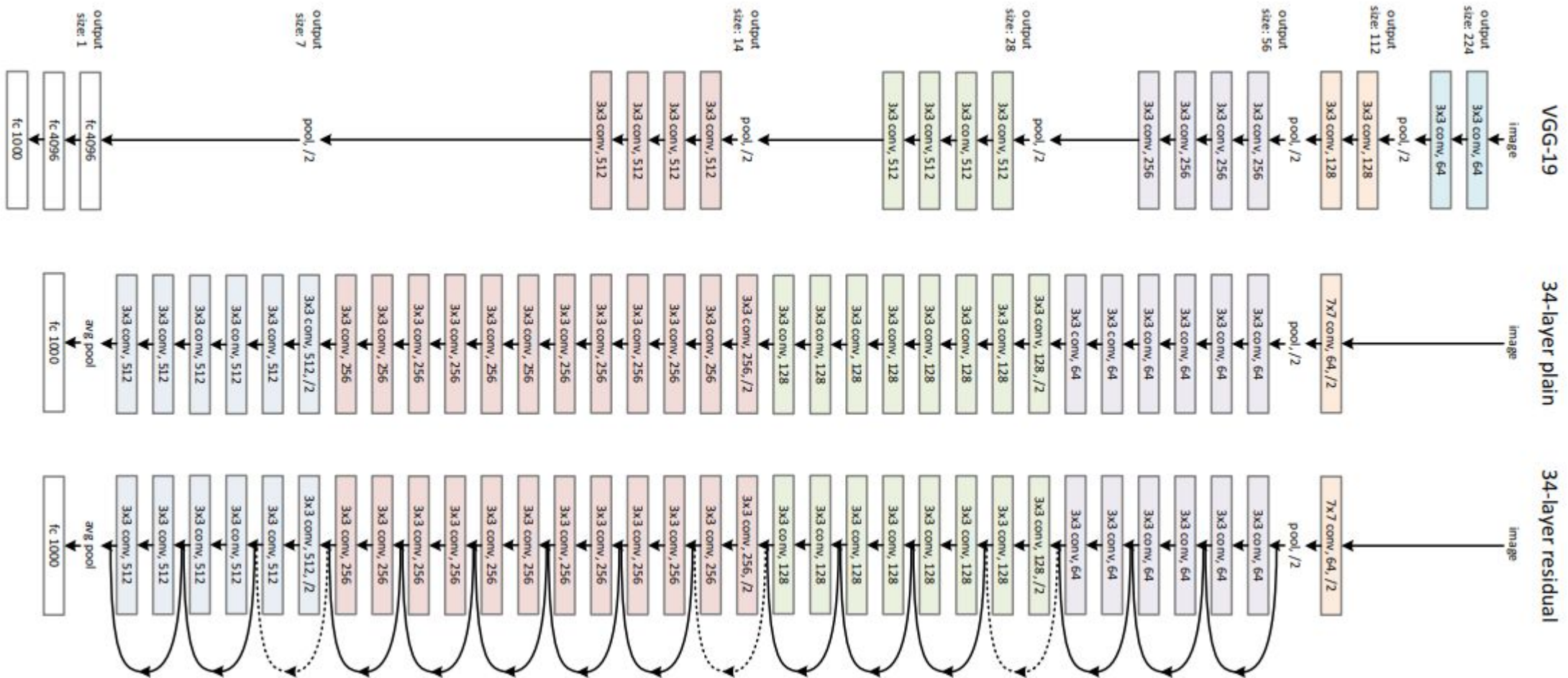
Benefits

Stable Forward Pass: Keeps variance growth linear instead of exponential.

Higher Learning Rates: Smoother gradients allow aggressive optimization.

Regularization: Noise from batch statistics acts like Dropout, reducing overfitting.

Creating Resnet using residual blocks



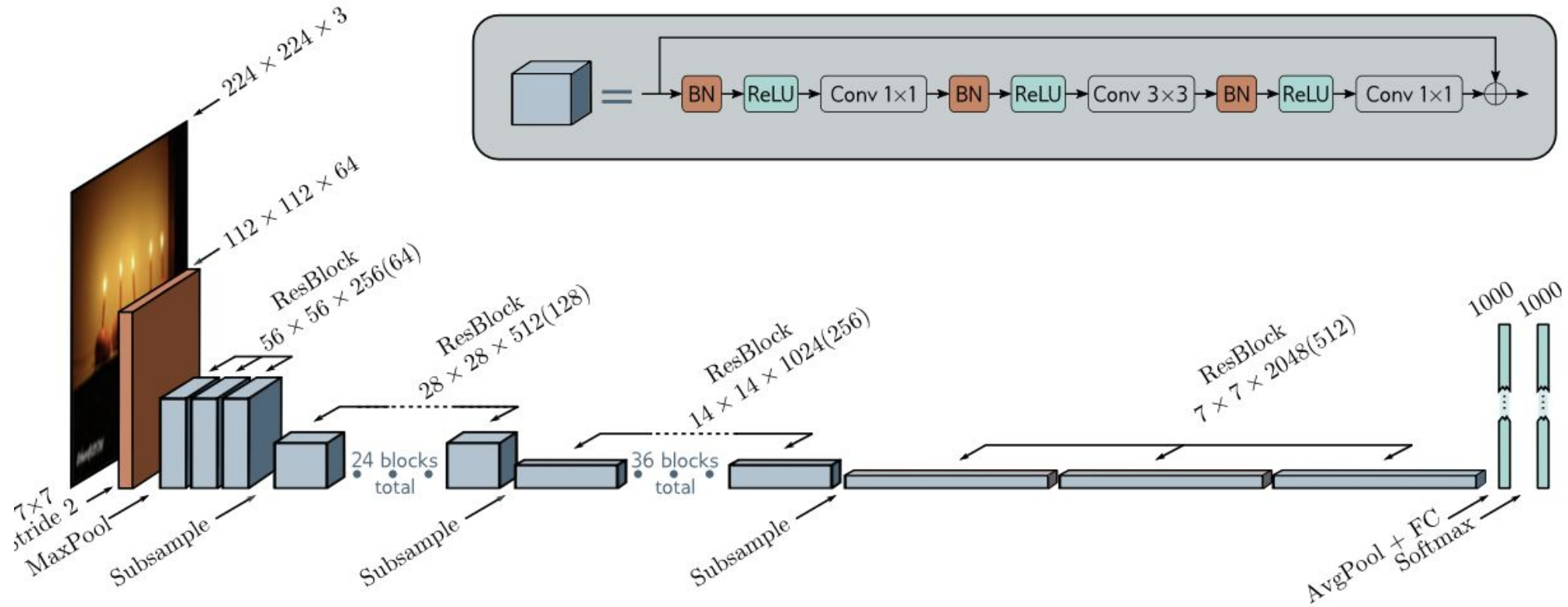
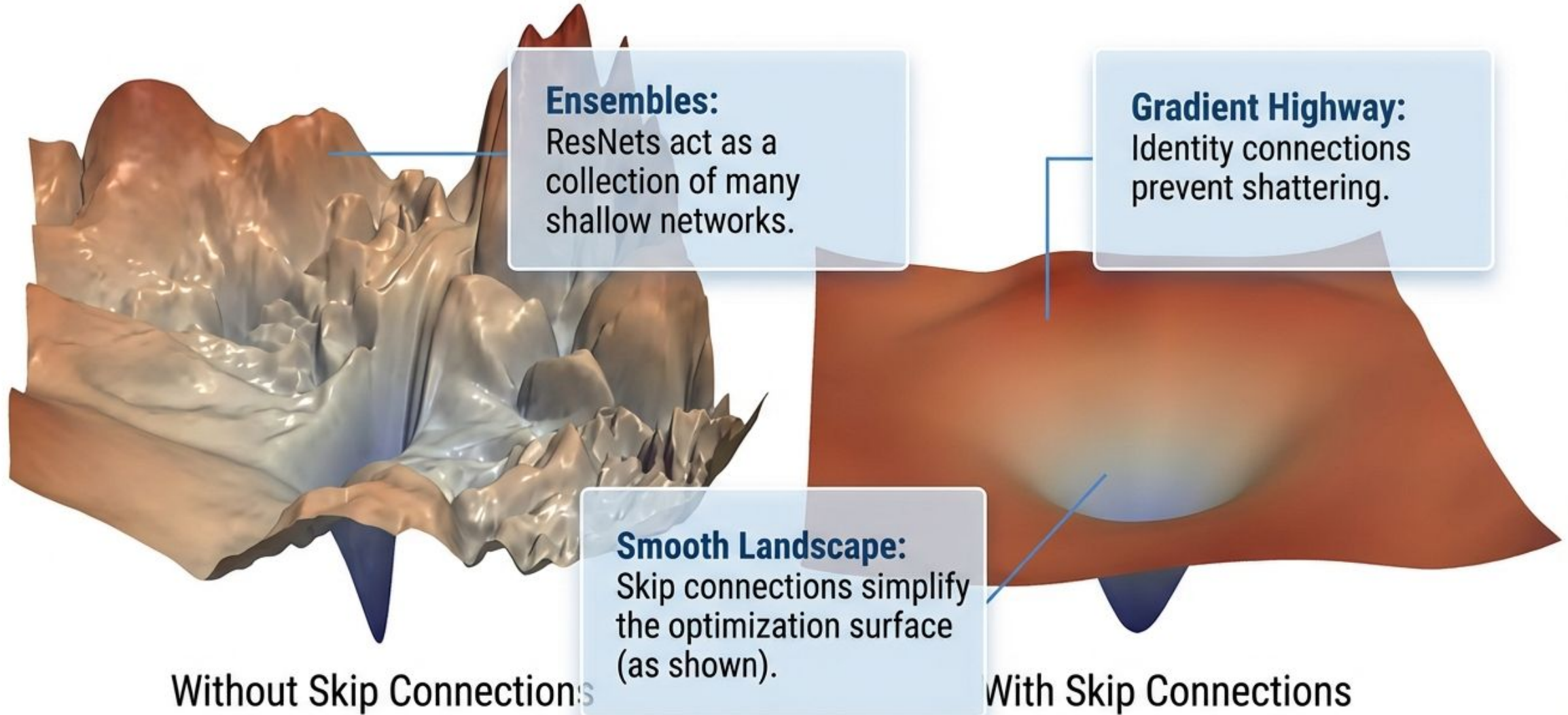


Figure 11.8 ResNet-200 model. A standard 7×7 convolutional layer with stride two is applied, followed by a MaxPool operation. A series of bottleneck residual blocks follow (number in brackets is channels after first 1×1 convolution), with periodic downsampling and accompanying increases in the number of channels. The network concludes with average pooling across all spatial positions and a fully connected layer that maps to pre-softmax activations.

Why ResNets Work: The Big Picture



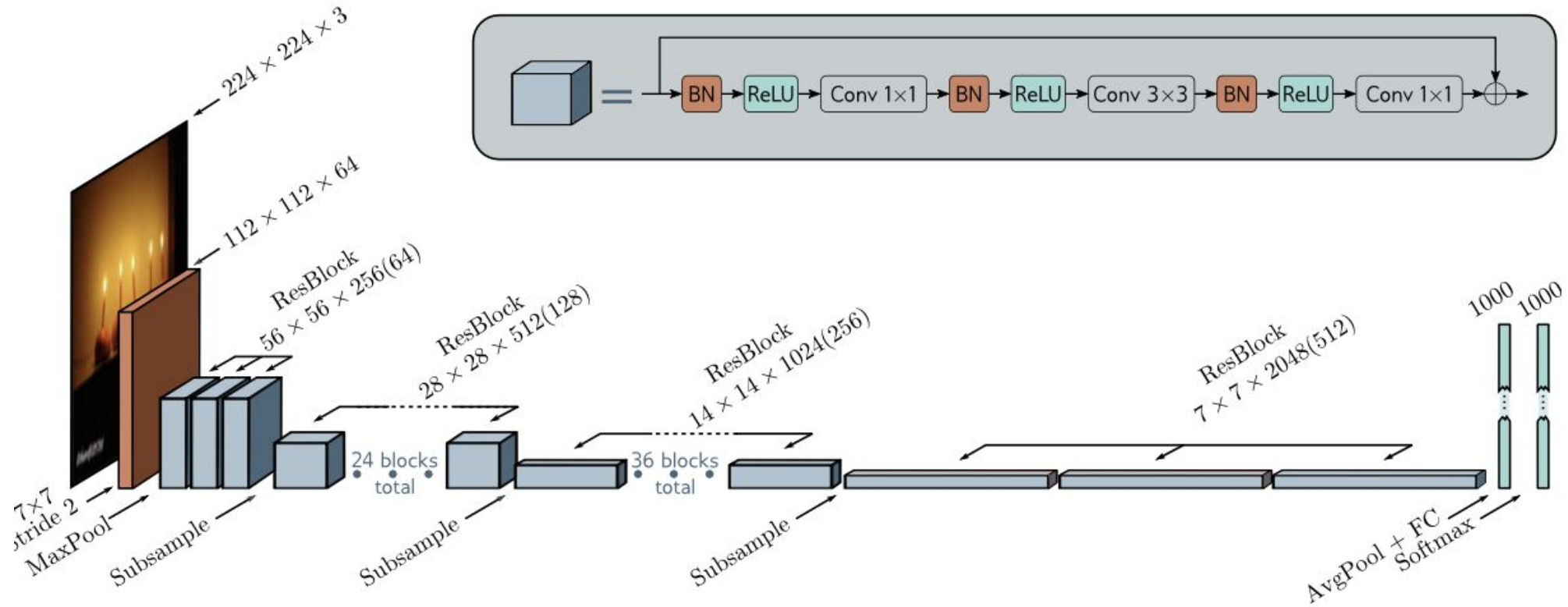
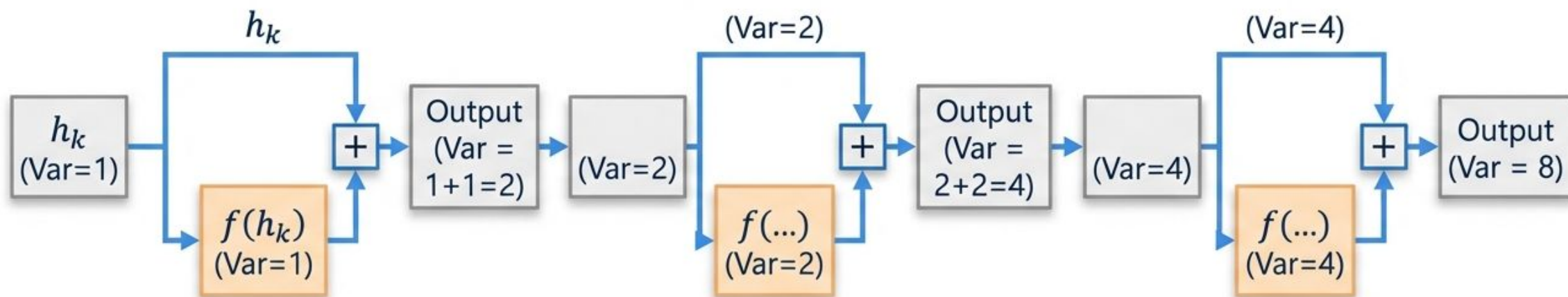


Figure 11.8 ResNet-200 model. A standard 7×7 convolutional layer with stride two is applied, followed by a MaxPool operation. A series of bottleneck residual blocks follow (number in brackets is channels after first 1×1 convolution), with periodic downsampling and accompanying increases in the number of channels. The network concludes with average pooling across all spatial positions and a fully connected layer that maps to pre-softmax activations.

Problem 11.4: Variance Explosion

Why does **variance increase** in residual blocks?



Assumption: Branches are uncorrelated ($Var(A + B) \approx Var(A) + Var(B)$).

Result: Exponential variance growth (2^K) requires Batch Normalization to stabilize.

Chapter 11 Summary: Key Takeaways



- **The Problem:** Deep sequential networks suffer from Shattered Gradients.



- **The Solution:** Residual Connections ($y = x + f(x)$) create gradient superhighways.



- **The Enabler:** Batch Normalization is essential to control exploding variance.



- **The Architectures:**
 - **ResNet:** The standard for classification.
 - **DenseNet:** Concatenates features for reuse.
 - **U-Net:** Uses long skips for segmentation.